

©2015 Acrome Inc., All rights reserved.

Acrome Inc.
ITU ARI4 Science Park
Maslak, Istanbul
Turkey
info@acrome.net
Phone: +90 532 132 17 22
Fax: +90 212 285 25 94

Printed in Maslak, Istanbul

For more information on the solutions Acrome Inc. offers, please visit the web site at:

<http://www.acrome.net>

This document and the software described in it are provided subject to a license agreement. Neither the software nor this document may be used or copied except as specified under the terms of that license agreement. All rights are reserved and no part may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of Acrome Inc.

ACKNOWLEDGEMENTS

Special thanks to Acrome myCONTROL Team for their supports in embedded control and preparation of the courseware.

Contents

1	Components of Delta Robot	3
1.1	Round Electromagnet	3
1.2	Smart Servo Motors	4
1.2.1	Overview	4
1.2.2	Theory of Operation	5
1.3	ACROME Power Distribution Box.....	6
1.4	Arduino Mega 2560	6
1.5	Digital Camera.....	7
2	Kinematics of Delta Robot	8
2.1	Forward Kinematics of Delta Robot	9
2.1.1	In-Lab Exercise: Forward Kinematics of Delta Robot	15
2.2	Inverse Kinematics of Delta Robot.....	17
2.2.1	In-Lab Exercise: Inverse Kinematics of Delta Robot	21
2.3	Mobility of Delta Robot.....	23
3	Trajectory Generation	24
3.1	Introduction	24
3.2	General Considerations in Path Description and Generation	24
3.3	Cartesian Space Schemes	26
3.4	Joint-Space Schemes.....	26
3.4.1	Cubic Polynomials	26
3.4.2	Cubic Polynomials for a path with via points	30
3.4.3	Higher Order Polynomials	33
3.4.4	Linear Function with Parabolic Blends	34
3.4.5	Linear function with parabolic blends for a path with via points	36
4	Vision Guidelines for Delta Robot	40
4.1	Introduction to Vision Systems	40
4.1.1	In-Lab Exercise: Vision Acquisition and Grayscale Processing	41
4.1.2	Threshold.....	42
4.1.3	Pattern Matching.....	43
4.2	Smash Application of Delta Robot.....	48
4.2.1	In-Lab Exercise: Smash Application	50
	References	52

1 Components of Delta Robot

All the main parts of “ACROME Delta Robot” can be seen below.

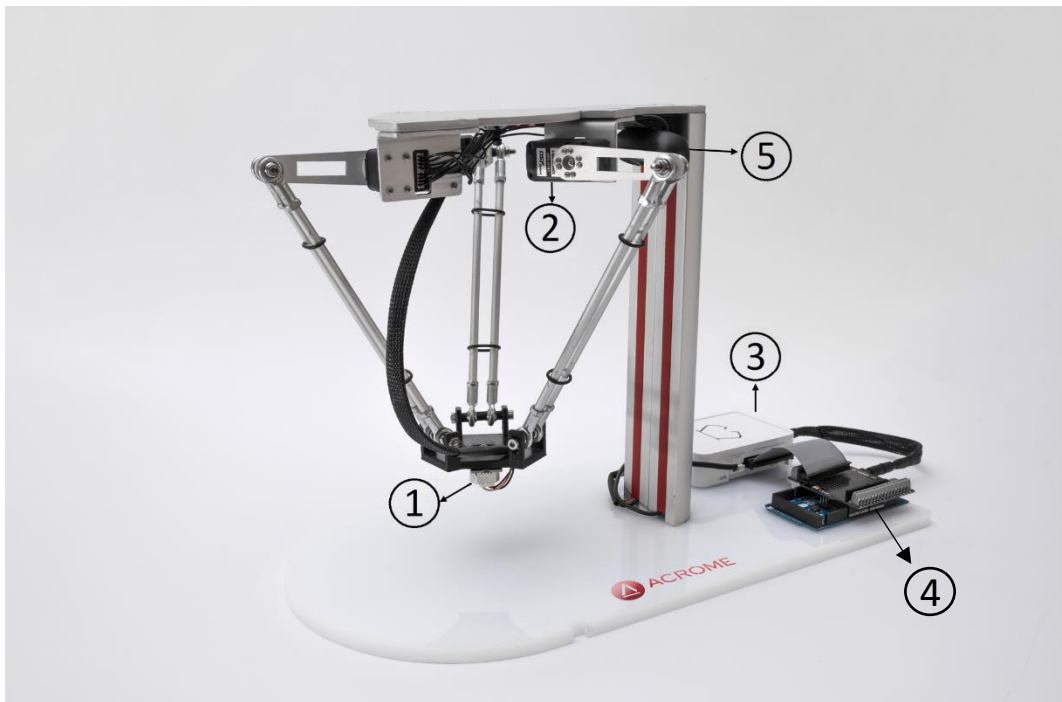


Figure 1.1: Components: 1. Smart Servo Motors 2. Round Electromagnet 3. ACROME Power Distribution Box 4. Arduino 5. Camera

1.1 Round Electromagnet

A round electromagnet is used for its ability to attract and hold ferromagnetic materials with varying degrees of force, through the application of a controlled DC electrical current. Electromagnets also have the ability to release the item when required. Electromagnets are commonly used in robotics and automated applications.

ZYE1-P20/15 type is chosen for pick and place operations of the delta robot. The holding force of a round electromagnet is determined as 25 N.

In this version of the Delta Robot, the round electromagnet is used to make a stylus pen tip. With this usage, the tablet screen detects the movement of the Delta Robot.

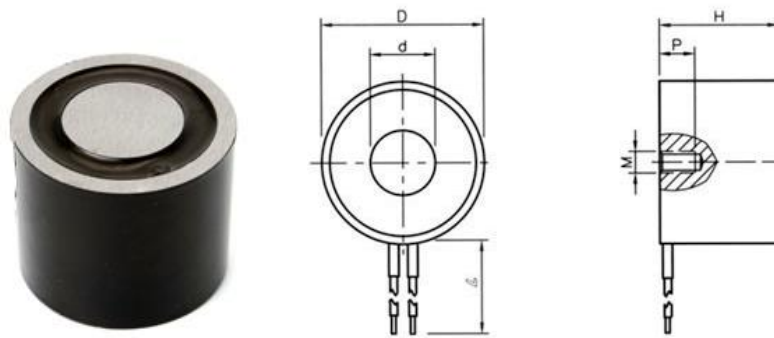


Figure 1.2: Round Electromagnet

Table 1.1: Technical Details for Round Electromagnet

Type	D (mm)	d (mm)	H (mm)	M (mm)	P (mm)	L (mm)	Power Consumption (W)	Holding Force (N)	Weight (g)
ZYE1-P20/15	20	8	15	M3	6	200	3	25 N	15 g

1.2 Smart Servo Motors

1.2.1 Overview

HerkuleX DRS 0101 is composed of a motor, a gear reducer, a control circuitry and a communications capability in one single package. These servos can be assembled easily with an added advantage of simple wiring. If required, each servo allows serial or parallel connections.



Figure 1.3: HerkuleX Smart Servo Motor

1.2.2 Theory of Operation

When the servo receives a movement command, it automatically creates a trapezoidal type speed profile which is shown in Figure 1.4 to control the position.

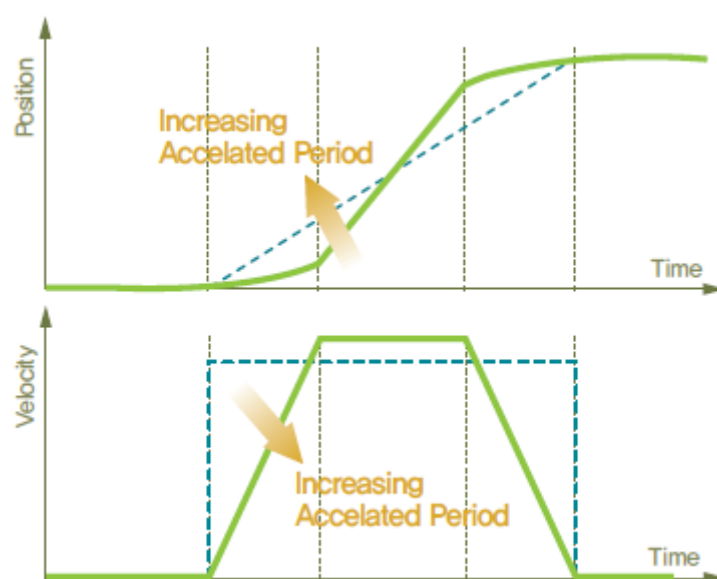


Figure 1.4: — Position and Trapezoidal Speed Profile

.... Position and Square Speed Profile

The trapezoidal type speed profile is chosen by default but the profile type could be changed to square or triangle profile type. While the smart servo is tracking the selected speed profile, it eliminates vibrations caused by the sudden acceleration and deceleration as found in the square type speed profile. In addition, it increases the energy efficiency while leading to smoother movement.

Table 1.2: Technical Specifications for HerkuleX DRS 0101

Specifications	
Stall Torque / Maximum Speed	12 kgf.cm
Reduction Ratio	1 : 266
Resolution	0.325°
Feedback	Position, Velocity, Load, Voltage, Temperature
Operation Angle	320°

1.3 ACROME Power Distribution Box

Three servo motors, round electromagnet and Arduino connections are located on the ACROME power distribution box which is shown in Figure 1.5. It also has a RGB led and switch mode regulators.

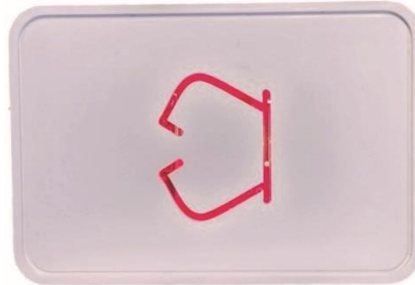


Figure 1.5: ACROME Power Distribution Box

1.4 Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller board based on the ATmega2560. It has 54 digital input/output pins (of which 15 can be used as PWM outputs), 16 analog inputs, 4 UARTs (hardware serial ports), a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button.



Figure 1.6: Arduino Mega 2560

1.5 Digital Camera

Logitech C310 HD 720p captures images for the purpose of determining the position of the desired object. Communication between computer and digital camera is provided with USB 2.0 port. Digital camera is compatible with Microsoft Windows 7, Windows 8 and Windows 10.



Figure 1.7: Digital Camera

2 Kinematics of Delta Robot

The ACROME Delta Robot is composed of three identical arms in parallel between the base plate and the travelling end-effector plate. The combination of the constrained motion of these arms results in 3 translational degrees of freedom. The upper robot arms are directly attached to the actuators. The three actuators are rigidly mounted on the base plate with 120° in between. Each of the lower arms have two parallel bars. It connects the upper arm with the travelling end-effector plate. The system was integrated with a round electromagnet and an USB camera for pick and place applications. The general mechanical structure of the ACROME Delta Root is shown in Figure 2.1.

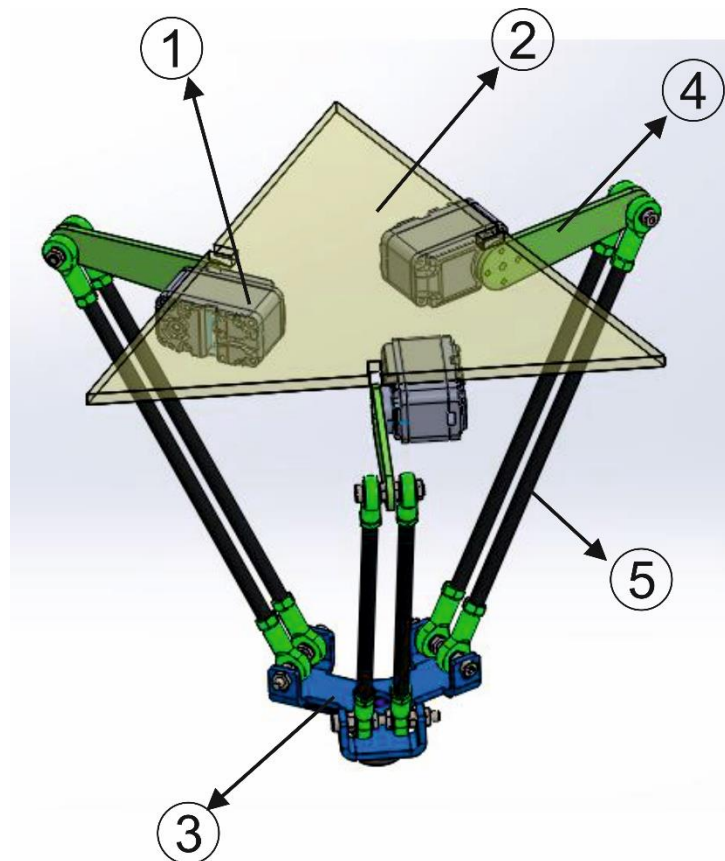


Figure 2.1: The General Mechanical Structure: 1. Smart Servo Motor 2. The base Plate
3. The Travelling End-effector Plate 4. The Upper Arm 5. The Lower Arm

In this chapter, we analyze the kinematics of Delta Robot including analytical solutions in two section: Forward and inverse kinematics. Delta Robot mechanism's kinematics analysis is simply the study of mapping between the Cartesian space and joint space. Mapping from the joint space to Cartesian space is called forward kinematics, while the mapping that is from Cartesian space to joint space is called inverse kinematics.

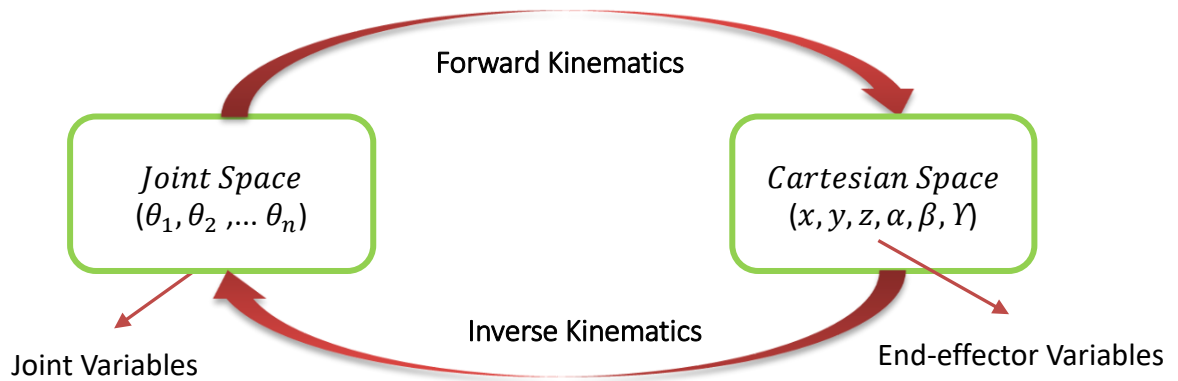


Figure 2.2: Relations between Forward and Inverse Kinematics

In the following section, we explain the forward kinematics of Delta Robot via geometric constructions at first, then we will obtain the inverse kinematics solution.

2.1 Forward Kinematics of Delta Robot

As the forward kinematics is shortly described above, the three joint angles $(\theta_1, \theta_2, \theta_3)$ are inputs and the Cartesian coordinates (x, y, z) are the outputs of the forward kinematics. Joint angles are applied to servo motors and the corresponding Cartesian coordinates are calculated via forward kinematics equations.

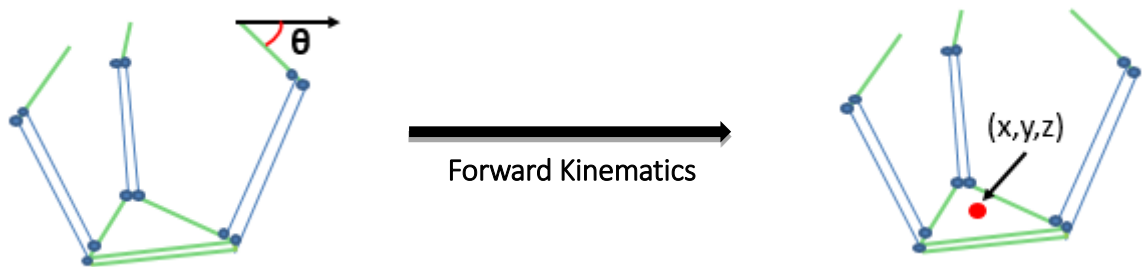
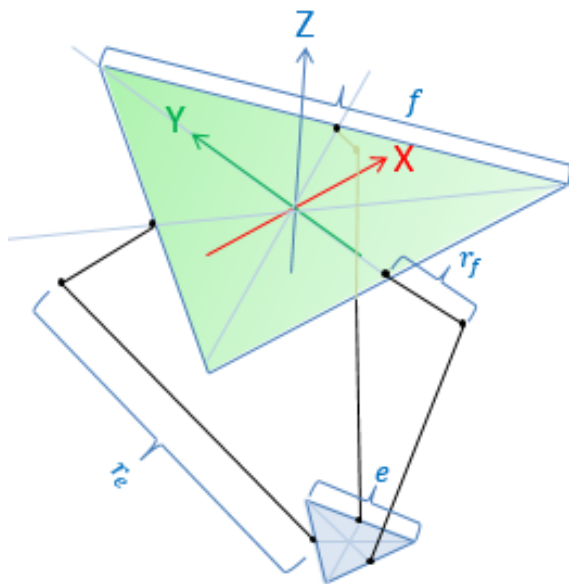


Figure 2.3 : Mapping from Joint Space to Cartesian Space

In general, the forward kinematics of parallel robots are difficult to compute but 3-DOF Delta Robot moves only translational. Therefore, a unique and correct analytical solution could be found in the solution set of the forward kinematics.

To construct the forward kinematics of the Delta Robot, let's first designate the physical parameters of the robot below.



Physical Dimensions of Delta Robot (mm)

f	Side of the base plate triangle	230.59
e	Side of the travelling plate triangle	112.96
r_f	Length of the upper arm	64.2
r_e	Length of the lower arm	200.0

Figure 2.4: Delta Robot Schematics

A reference will be chosen at the center of the base triangle so z-coordinate of the end-effector will always be **negative**. It is useful for choosing the unique and correct solution in the forward kinematics solution set.

To obtain the forward kinematics solution, first we need to find the coordinates of points J_1, J_2, J_3 from the joint angles $\theta_1, \theta_2, \theta_3$. While the lower arms are rotating around points J_1, J_2, J_3 , we will obtain three spheres with Radius r_e . To intersect these spheres in one point, we move the centers of the spheres from points J_1, J_2, J_3 to points J'_1, J'_2, J'_3 . The points and angles described above are shown in Figure 2.5.

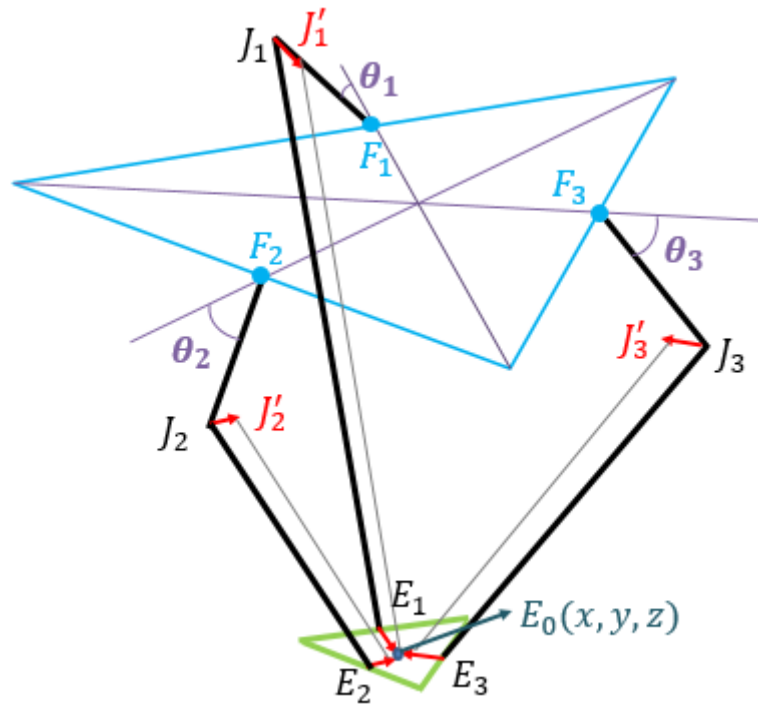


Figure 2.5: Delta Robot Links, Vertices and Frame Representations

The intersection point of all three spheres is the point $E_0(x, y, z)$ and this intersection point is the Cartesian space solution of the forward kinematics.

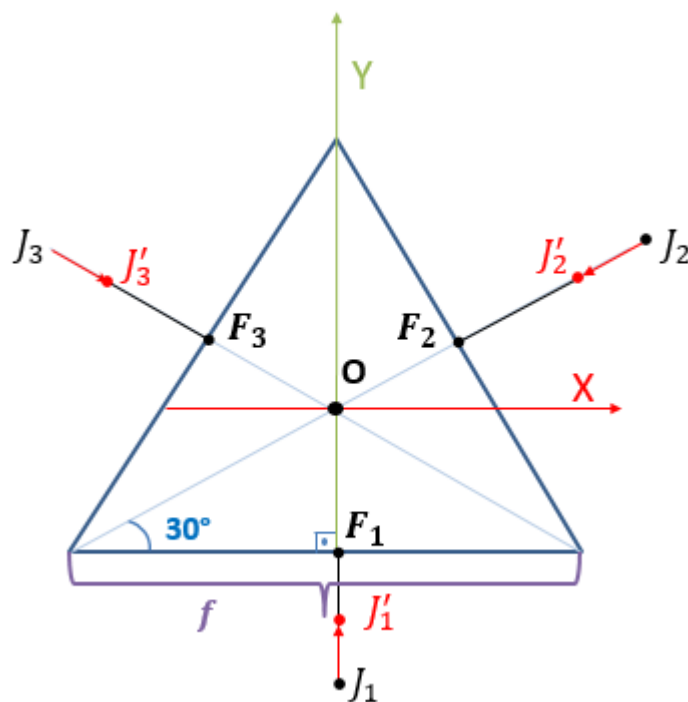


Figure 2.6: The Triangle of Base Plate

To find the intersection point, the center and the radius of each sphere must be found. The triangle of the base plate is shown in Figure 2.6 and we define the length of the side of this triangle as f before. f is the known parameter so we can calculate the distances between points F_i and the center of the base triangle O .

$$|OF_1| = |OF_2| = |OF_3| = \frac{f}{2} \tan 30 \quad (2.1)$$

Similarly, the distances between points J_i and points J'_i are as follows:

$$|J_1J'_1| = |J_2J'_2| = |J_3J'_3| = \frac{e}{2} \tan 30 \quad (2.2)$$

And the distances between points F_i and points J_i can be obtained as:

$$|F_1J_1| = r_f \cos \theta_1, \quad |F_2J_2| = r_f \cos \theta_2, \quad |F_3J_3| = r_f \cos \theta_3 \quad (2.3)$$

If we combine the Equations 1, 2 and 3, we obtain the centers of the all three spheres with radius r_e respectively below.

$$\begin{aligned} \left\{ \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix}, r_1 \right\} &= \left\{ \begin{bmatrix} 0 \\ -\left(\frac{f-e}{2} \tan 30\right) - r_f \cos \theta_1 \\ -r_f \sin \theta_1 \end{bmatrix}, r_e \right\} \\ \left\{ \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix}, r_2 \right\} &= \left\{ \begin{bmatrix} \left(\left(\frac{f-e}{2} \tan 30\right) + r_f \cos \theta_2\right) \cos 30 \\ \left(\left(\frac{f-e}{2} \tan 30\right) + r_f \cos \theta_2\right) \sin 30 \\ -r_f \sin \theta_2 \end{bmatrix}, r_e \right\} \\ \left\{ \begin{bmatrix} x_3 \\ y_3 \\ z_3 \end{bmatrix}, r_3 \right\} &= \left\{ \begin{bmatrix} \left(\left(\frac{f-e}{2} \tan 30\right) + r_f \cos \theta_3\right) \cos 30 \\ -\left(\left(\frac{f-e}{2} \tan 30\right) + r_f \cos \theta_3\right) \sin 30 \\ -r_f \sin \theta_3 \end{bmatrix}, r_e \right\} \end{aligned} \quad (2.4)$$

The equation of a sphere with radius r_e , centered at the below points are given in Cartesian coordinates by the following equations:

$$\begin{aligned} x^2 + (y - y_1)^2 + (z - z_1)^2 &= r_e^2 \\ (x - x_2)^2 + (y - y_2)^2 + (z - z_2)^2 &= r_e^2 \\ (x - x_3)^2 + (y - y_3)^2 + (z - z_3)^2 &= r_e^2 \end{aligned} \quad (2.5)$$

Let's expand the above three equations by squaring all left side terms and they can be written as:

$$x^2 + y^2 + z^2 - 2y_1y - 2z_1z = r_e^2 - y_1^2 - z_1^2 \quad (2.6)$$

$$x^2 + y^2 + z^2 - 2x_2x - 2y_2y - 2z_2z = r_e^2 - x_2^2 - y_2^2 - z_2^2 \quad (2.7)$$

$$x^2 + y^2 + z^2 - 2x_3x - 2y_3y - 2z_3z = r_e^2 - x_3^2 - y_3^2 - z_3^2 \quad (2.8)$$

To simplify the above equations, substitute $w_i = x_i^2 + y_i^2 + z_i^2$ into the Equation 2.6, 2.7 and 2.8, and we can rewrite the equations as below:

$$x^2 + y^2 + z^2 - 2y_1y - 2z_1z = r_e^2 - w_1 \quad (2.9)$$

$$x^2 + y^2 + z^2 - 2x_2x - 2y_2y - 2z_2z = r_e^2 - w_2 \quad (2.10)$$

$$x^2 + y^2 + z^2 - 2x_3x - 2y_3y - 2z_3z = r_e^2 - w_3 \quad (2.11)$$

Then, to eliminate $x^2 + y^2 + z^2$, subtract the Equation 2.10 and 2.11 from the first equation in 2.9. As a result, we have the new equation sets respectively:

$$x_2x + (y_1 - y_2)y + (z_1 - z_2)z = \frac{w_1 - w_2}{2} \quad (2.12)$$

$$x_3x + (y_1 - y_3)y + (z_1 - z_3)z = \frac{w_1 - w_3}{2} \quad (2.13)$$

Now, to equal the coefficients of x terms, Equation 2.12 multiplies by x_3 and Equation 2.13 multiplies by x_2 :

$$x_2x_3x + x_3(y_1 - y_2)y + x_3(z_1 - z_2)z = x_3 \left(\frac{w_1 - w_2}{2} \right) \quad (2.14)$$

$$x_2x_3x + x_2(y_1 - y_3)y + x_2(z_1 - z_3)z = x_2 \left(\frac{w_1 - w_3}{2} \right) \quad (2.15)$$

To eliminate x_2x_3x terms, subtract Equation 2.14 from the Equation 2.15. We obtain two equations:

$$x = \frac{a_1z + b_1}{d} \quad (2.16)$$

$$y = \frac{a_2z + b_2}{d} \quad (2.17)$$

And to simplify the equations, we can describe the new denominator parameter in 2.18:

$$d = x_3(y_2 - y_1) - x_2(y_3 - y_1) \quad (2.18)$$

Finally, we obtain a_1, b_1, a_2, b_2 parameters used in Equations 2.16 and 2.17 as below:

$$a_1 = (z_1 - z_2)(y_3 - y_1) - (z_3 - z_1)(y_2 - y_1) \quad (2.19)$$

$$b_1 = -0.5 [(w_2 - w_1)(y_3 - y_1) - (w_3 - w_1)(y_2 - y_1)] \quad (2.20)$$

$$a_2 = (z_2 - z_1)x_3 - (z_3 - z_1)x_2 \quad (2.21)$$

$$b_2 = 0.5 [(w_2 - w_1)x_3 - (w_3 - w_1)x_2] \quad (2.22)$$

Until this point, we have tried to find the expression of x and y in terms of z so we have Equations 2.16 and 2.17. Now, the expressions obtained for x and y are substituted in Equation 2.6 to eliminate x and y :

$$a = (a_1^2 + a_2^2 + d^2) \quad (2.23)$$

$$b = 2(a_1b_1 + a_2(b_2 - dy_1) - z_1d^2) \quad (2.24)$$

$$c = b_1^2 + (b_2 - dy_1)^2 + d^2(z_1^2 - r_e^2) \quad (2.25)$$

$$az^2 + bz + c = 0 \quad (2.26)$$

As you can see, Equation 2.26 is a second-order linear equation. If the discriminant is bigger than zero, there are two solutions for z .

$$\Delta = b^2 - 4ac > 0 \quad \rightarrow \quad z_{\pm} = \frac{-b \pm \sqrt{\Delta}}{2a} \quad (2.27)$$

As mentioned before, we choose the negative z solution because the origin point is located on the base plate. Therefore, downward direction has a negative sign. The negative solution for z is:

$$z = \frac{-b + \sqrt{\Delta}}{2a} \quad (2.28)$$

To complete the forward kinematics solution of the Delta Robot, we substitute z from 2.28 into 2.16 and 2.17. Finally, we have one solution:

$$\{x \ y \ z\}^T \quad (2.29)$$

2.1.1 In-Lab Exercise: Forward Kinematics of Delta Robot

1. Open the “ForwardKinematicsforStudent.slx” under the Delta Robot Project.

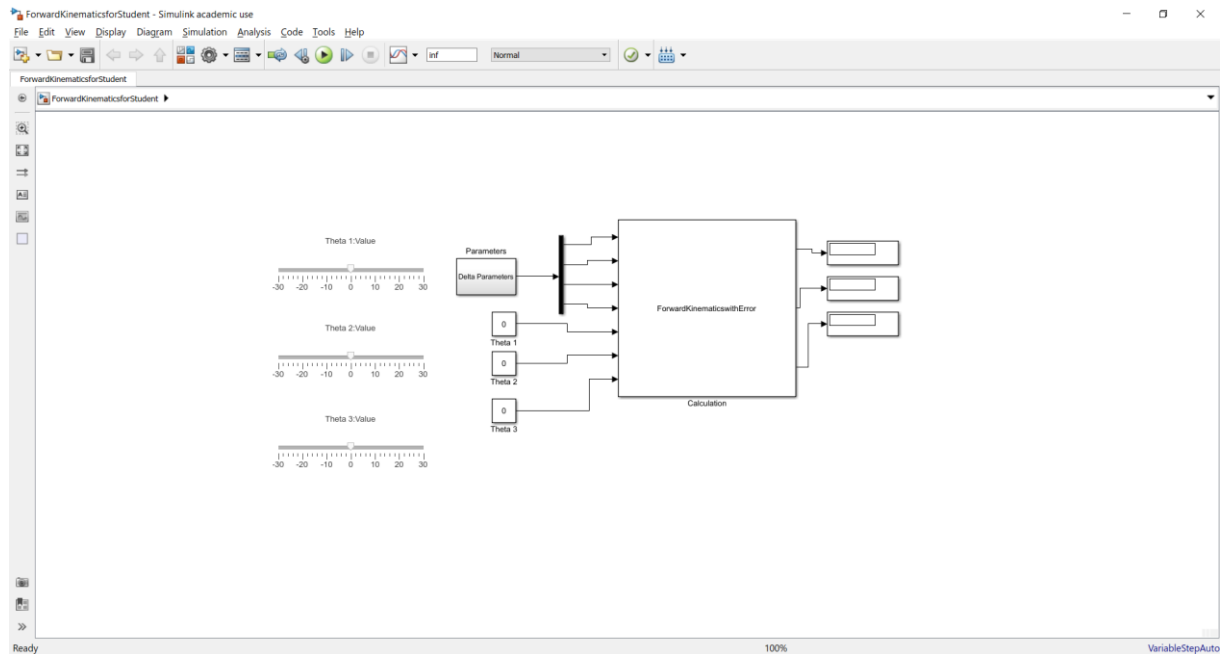


Figure 2.7: The Block Diagram of “ForwardKinematicsforStudent.slx”

2. You will see that the Run button is on block diagram toolbar.



Figure 2.8: Block Diagram Toolbar

3. Now, click the Run button. Diagnostic Viewer window will appear and will list all errors. Check the errors and warnings. You can click the error from the list (written and underlined with blue) to navigate MATLAB code

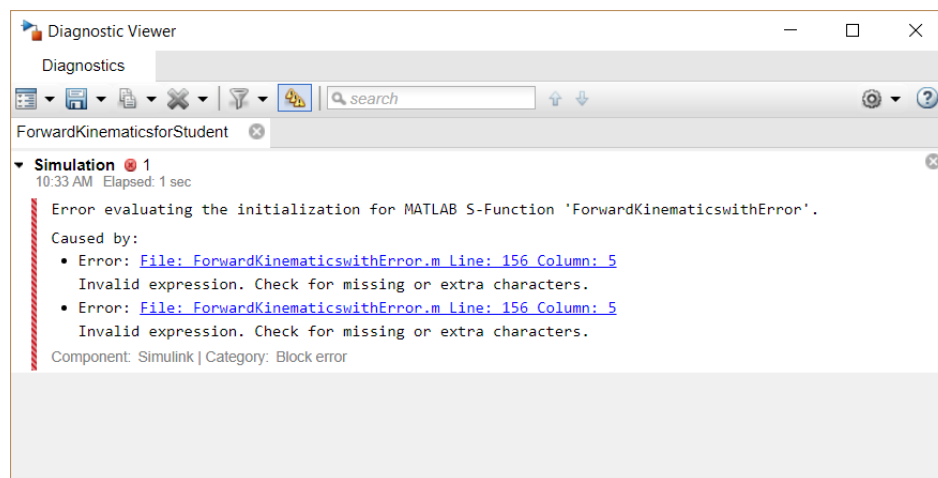
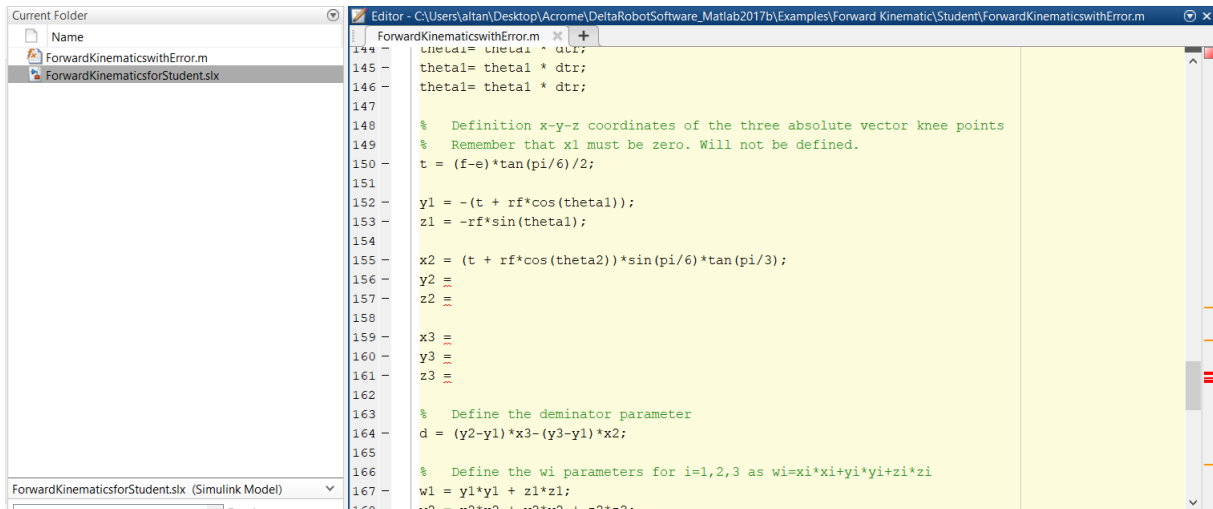


Figure 2.9: Error List Window

- After clicking, you will see the MATLAB code of “ForwardKinematicswithError.m” as in Figure 2.10.



```

144 thetai= thetai * dtr;
145 thetai= thetai * dtr;
146 thetai= thetai * dtr;
147
148 % Definition x-y-z coordinates of the three absolute vector knee points
149 % Remember that x1 must be zero. Will not be defined.
150 t = (f-e)*tan(pi/6)/2;
151
152 y1 = -(t + rf*cos(theta1));
153 z1 = -rf*sin(theta1);
154
155 x2 = (t + rf*cos(theta2))*sin(pi/6)*tan(pi/3);
156 y2 =
157 z2 =
158
159 x3 =
160 y3 =
161 z3 =
162
163 % Define the denominator parameter
164 d = (y2-y1)*x3-(y3-y1)*x2;
165
166 % Define the wi parameters for i=1,2,3 as wi=xi*xi+yi*yi+zi*zi
167 w1 = y1*y1 + z1*z1;
168 w2 = x2*x2 + y2*y2 + z2*z2;
169 w3 = x3*x3 + y3*y3 + z3*z3;

```

Figure 2.10: The MATLAB code of “ForwardKinematicswithError.m”

- To run the program, fill in the blanks in MATLAB code. The color of the wrong lines in MATLAB code is red. All expressions for the codes are explained in Chapter 2.1.
- To check the defined variables, we have three different motor angles and the corresponding coordinates of these angles. Run the Simulink model and enter the angles respectively. When the listed angle values are entered the Parameters of the Delta Robot, you should read the below x, y and z values.

$$\begin{aligned}
 \{15,30,30\} &\rightarrow \{0, -23.8995, -203.18\} \\
 \{10,20,45\} &\rightarrow \{36.029, -35.9022, -198.089\} \\
 \{0,30,0\} &\rightarrow \{-38.1122, -22.0041, -180.965\}
 \end{aligned}$$

- Now, “ForwardKinematicsforsStudent.slx” is ready to use.

2.2 Inverse Kinematics of Delta Robot

To know the desired position of the end-effector, we need to determine the angle of each joint to set motors in the proper position for picking. This process is known as **inverse kinematics**, in the following chapter we will explain the inverse kinematic solutions for the Delta Robot.

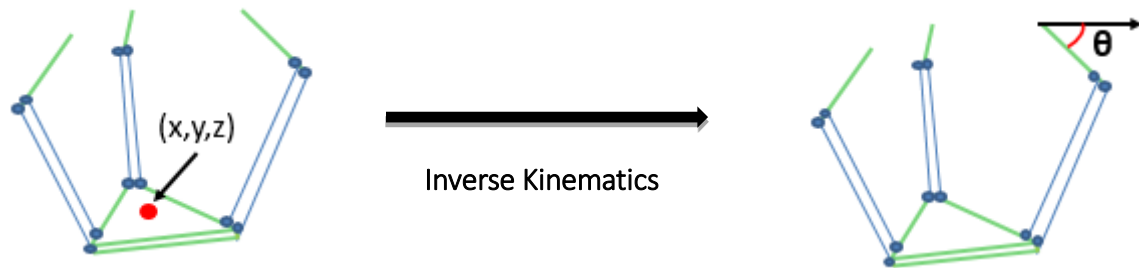


Figure 2.11: Mapping from Cartesian Space to Joint Space

To construct the inverse kinematics of the delta robot, let's return to Figure 2.4 and remember the Delta Robot's physical parameters. Like the forward kinematics section of the Delta Robot, we will move step-by-step.

The $F_1 J_1$ joint can rotate in YZ plane forming circle with center in point F_1 with radius r_f due to the robot design. As opposed to F_1, J_1 and E_1 are called universal joints which mean that $E_1 J_1$ can rotate freely relative to E_1 , forming a sphere with the center at point E_1 and radius r_e .

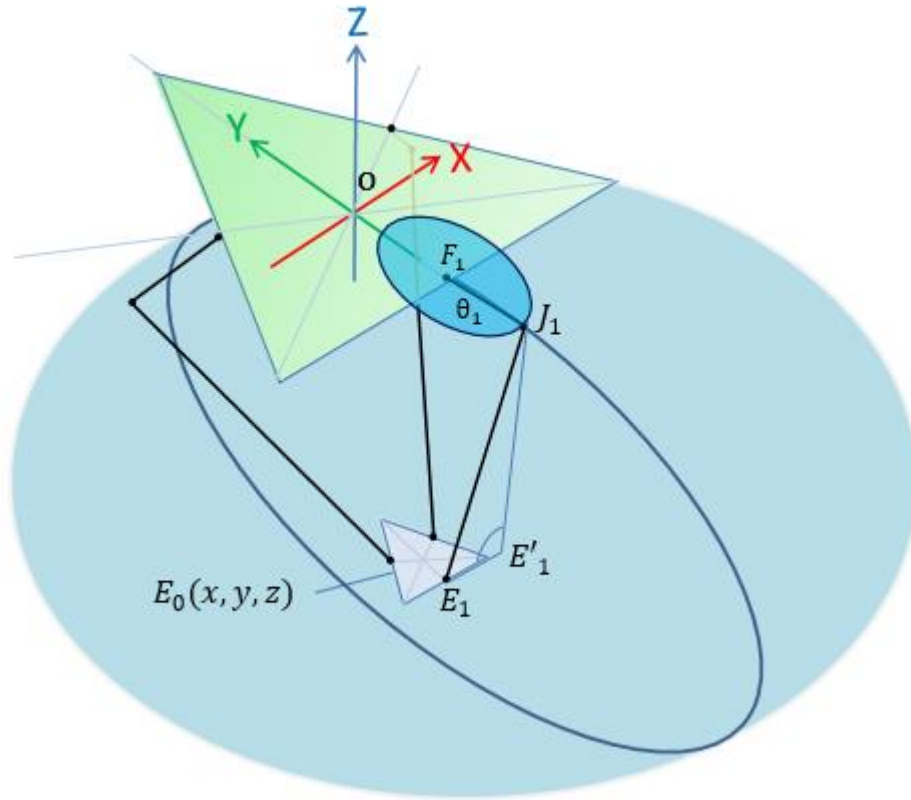


Figure 2.12: Sample Joint Path

Intersection of this sphere and the YZ plane is a circle with its center at the point E'_1 and radius $E'_1 J_1$, where E'_1 is the projection of the point E_1 on the YZ plane. The point J_1 can be found now as the intersection of the two circles with known radius' and centers at E'_1 and F_1 , and we can calculate θ_1 angle.

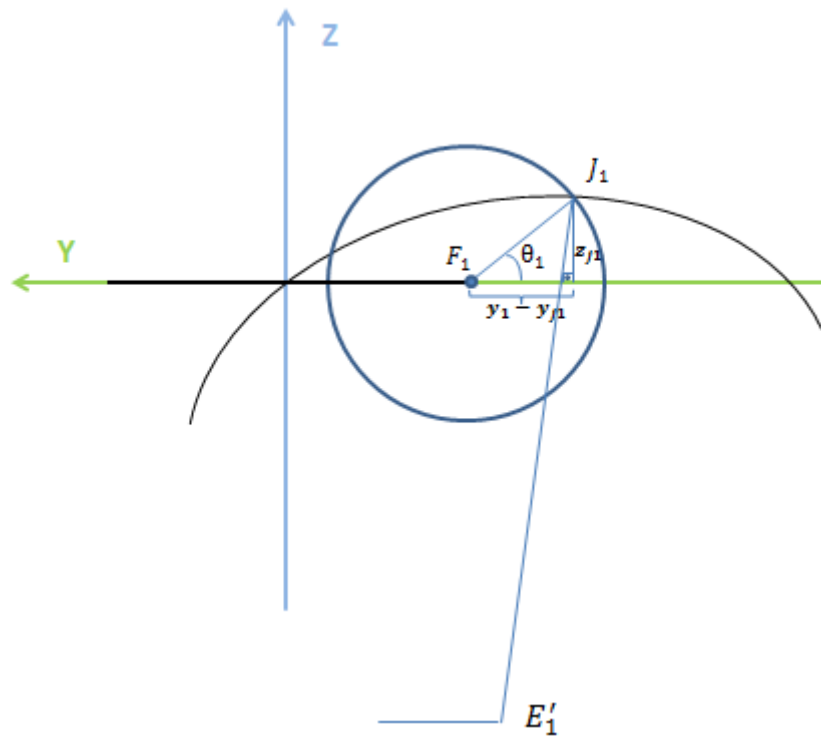


Figure 2.13: Cartesian Space Representation of Sample Joint

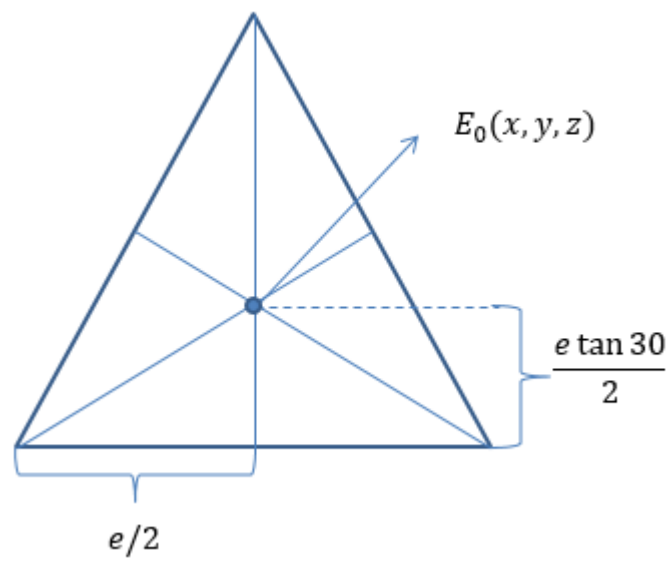


Figure 2.14: Triangle of the Travelling Plate

So the corresponding equations can be found and the YZ plane can be seen in the **above** figure:

$E_0(x, y, z)$ is obtained by subtracting the travelling plate's coordinates from the center of the base triangle's coordinates:

$$EE_1 = \frac{e \tan \pi/6}{2} = \frac{e}{2\sqrt{3}} \quad 2.30$$

$$E_1 \left(x, y - \frac{e}{2\sqrt{3}}, z \right) \rightarrow E'_1 \left(0, y - \frac{e}{2\sqrt{3}}, z \right) \quad 2.31$$

$$E_1 E'_1 = x \rightarrow E'_1 J_1 = \sqrt{E_1 J_1^2 - E_1 E'_1^2} = \sqrt{r_e^2 - x^2} \quad 2.32$$

Coordinates of point F_1 is as follows:

$$F_1 \left(0, \frac{-f}{2\sqrt{3}}, 0 \right) \quad 2.33$$

Constructing Euclidean equations from Figure 2.13, we obtain:

$$(y_{J1} - y_{F1})^2 + (z_{J1} - z_{F1})^2 = r_f^2 \rightarrow \left(y_{J1} + \frac{f}{2\sqrt{3}} \right)^2 + (z_{J1})^2 = r_f^2 \quad 2.3$$

$$(y_{J1} - y_{E'_1})^2 + (z_{J1} - z_{E'_1})^2 = r_e^2 - x^2 \rightarrow \left(y_{J1} - y + \frac{e}{2\sqrt{3}} \right)^2 + (z_{J1} - z)^2 = r_e^2 \quad 4$$

Finally, inverse kinematics solution for the first joint is:

$$\theta_1 = \arctan \left(\frac{z_{J1}}{y_{F1} - y_{J1}} \right) \quad 2.35$$

For remaining angles θ_2, θ_3 , let's rotate the coordinate system in the XY plane around z-axis by 120 and -120 degrees.

$$\left. \begin{aligned} x' &= x \cos \left(\frac{2\pi}{3} \right) + y \sin \left(\frac{2\pi}{3} \right) \\ y' &= -x \sin \left(\frac{2\pi}{3} \right) + y \cos \left(\frac{2\pi}{3} \right) \end{aligned} \right\} E_0(x, y, z) \rightarrow (x', y', z) \quad 2.36$$

2.2.1 In-Lab Exercise: Inverse Kinematics of Delta Robot

1. Open the “InverseKinematicsforStudent.slx” where its front panel is shown in Figure 2.15.

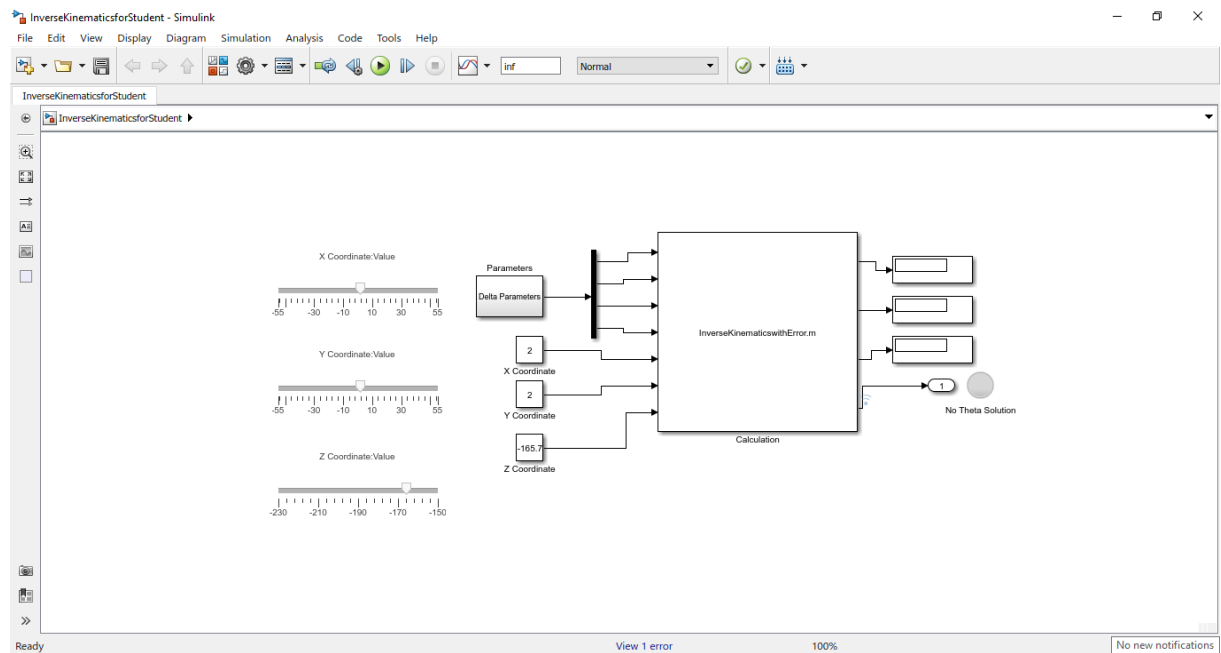


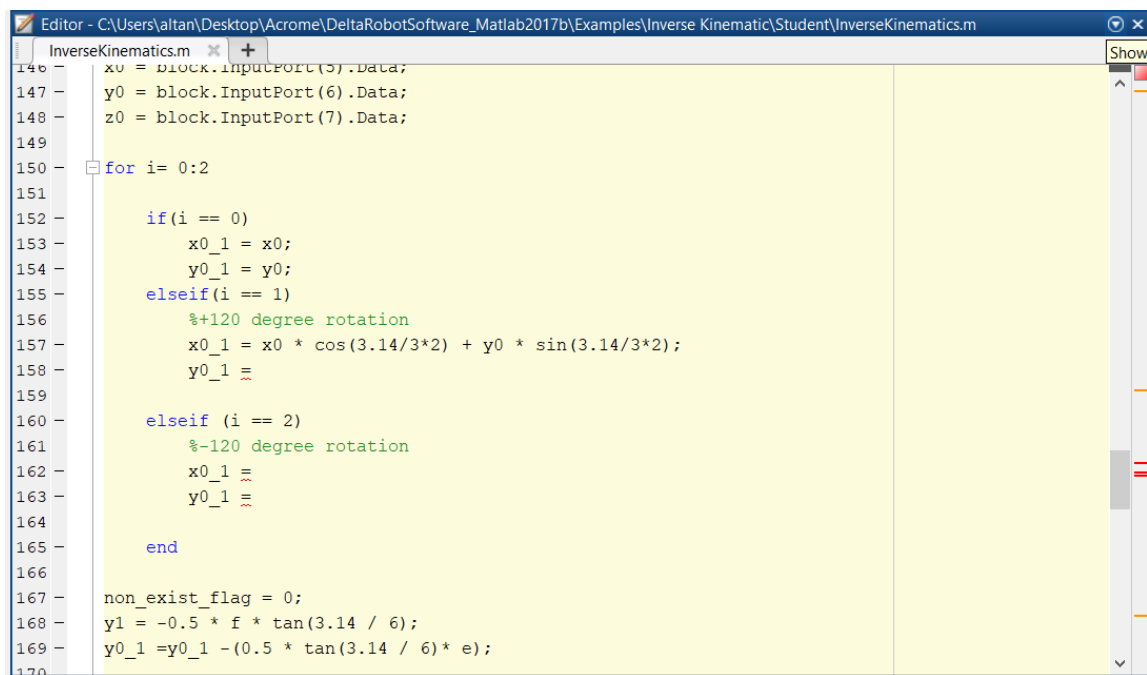
Figure 2.15: The Block Diagram of “InverseKinematicsforStudent.slx”

2. Now, click the Run button. Diagnostic Viewer window will appear and list all errors. Check the errors and warnings. You can click the error from the list (written and underlined with blue) to navigate MATLAB code.

```
Error evaluating the initialization for MATLAB S-Function 'InverseKinematicswithError'.  
Caused by:  
• Error: File: InverseKinematicswithError.m Line: 158 Column: 15  
  Expression or statement is incomplete or incorrect.  
• Error: File: InverseKinematicswithError.m Line: 158 Column: 15  
  Expression or statement is incomplete or incorrect.  
Component: Simulink | Category: Block error
```

Figure 2.16: Diagnostic Viewer

3. Fill the red undefined variables in the MATLAB code according to Equation 2.36. To check defined variables, use the same angles and coordinates given in 2.1.1. In-Lab Exercise Step 6.



```
146 - x0 = block.InputPort(5).Data;
147 - y0 = block.InputPort(6).Data;
148 - z0 = block.InputPort(7).Data;
149
150 - for i= 0:2
151
152 -     if(i == 0)
153 -         x0_1 = x0;
154 -         y0_1 = y0;
155 -     elseif(i == 1)
156 -         %+120 degree rotation
157 -         x0_1 = x0 * cos(3.14/3*2) + y0 * sin(3.14/3*2);
158 -         y0_1 =
159
160 -     elseif (i == 2)
161 -         % -120 degree rotation
162 -         x0_1 =
163 -         y0_1 =
164
165 -     end
166
167 - non_exist_flag = 0;
168 - y1 = -0.5 * f * tan(3.14 / 6);
169 - y0_1 = y0_1 - (0.5 * tan(3.14 / 6) * e);
170
```

Figure 2.17: MATLAB code of “InverseKinematicswithError.m”

4. At this point, “InverseKinematicsforStudent.slx” is ready to use

2.3 Mobility of Delta Robot

To show the degree-of-freedom of the Delta robot, we can use Kutzbach mobility equation which is:

$$M = 6(N - 1) - 5J_1 - 4J_2 - 3J_3 \quad 2.37$$

Table 2.1: Technical Details for Round Electromagnet

M	Mobility, # degree of freedom	
N	Total number of links, including ground	
J ₁	# one DOF joints	Revolute & prismatic joints
J ₂	# two DOF joints	Universal joints
J ₃	# three DOF joints	Spherical joints

We ignore the four-parallel mechanism by replacing each with a single link instead and we count a universal joint at either end of this virtual link.

$$N = 8, \quad J_1 = 3, \quad J_2 = 6, \quad J_3 = 0 \rightarrow M = 3 \text{ DOF} \quad 2.38$$

3 Trajectory Generation

3.1 Introduction

In general, a trajectory or path describes the desired motion of a manipulator in multidimensional space. In robotics, trajectory defines as a time history of position, velocity, and acceleration for each degree of freedom. Here, the basic problem is trajectory generation which refers to how to design the trajectory in joint space from the initial point to the final point. In order to solve this problem, we assume that we can compute the desired joint angle for the final point of the end effector. In other words, each point is converted into a set of desired joint angles, $\theta(t)$. This conservation is called inverse kinematics.

Trajectory generation occurs at run time; in the most general case, position, velocity, and acceleration are computed. These trajectories are computed on computers in discrete time so the trajectory points are computed at a certain rate, called the path-update rate. In typical manipulator systems, this rate lies between 60 and 2000 Hz.

3.2 General Considerations in Path Description and Generation

Motions of manipulator are described as motions of the tool frame, $\{T\}$, relative to the station frame, $\{S\}$. The aim of the motion is moving the manipulator from an initial position to some desired final position stated in other words moving the tool frame from its current value, $\{T_{initial}\}$, to a desired final value, $\{T_{final}\}$.

The motion between the tool frame and the station frame consists of change in orientation and position. The current value and final value of tool frame, via point and stationary frame is shown in Figure 3.1.

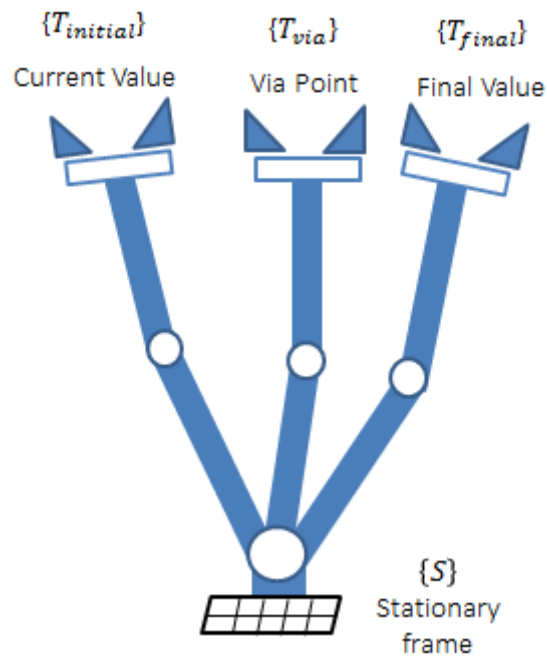


Figure 3.1: Tool Frame and Stationary Frame of the Manipulator

It is necessary to give more details to specify the motion and simply stating the desired final configuration. First method is giving a sequence of desired via points that are intermediate points between the initial and final positions. In this way, the tool frame moves through a set of intermediate positions and orientations described as via points. Path points consist of final, initial and all via points. Each of these points are actually a frame that specifies both the position and orientation of the tool relative to the station.

There are three constraints on temporal attributes of the motion such as the time elapsed between via points, the smoothness of the path and the spatial constraint. In order to guarantee smooth paths, we must put some sort of constraints on the spatial and temporal qualities of the path between via points. We will define a smooth function. A smooth function is continuous and has continuous first and second derivative functions. At this point, there are many choices for a path to be specific and planned.

Trajectory planning can be verified by two different ways: joint space and Cartesian space schemes. In the following chapter, we will define these schemes and explain the differences between these.

3.3 Cartesian Space Schemes

In Cartesian space, path generation is described in terms of functions of x , y and z coordinates. The joint angles are repeatedly calculated through inverse kinematics equations of the Delta Robot. As mentioned before, the Cartesian reference frame of the Delta Robot is the center of the base plate triangle. Therefore, Cartesian space trajectories moves relative to this reference frame.

Each axis moves independently so there are three trajectories for the Delta Robot. There are advantages and disadvantages for Cartesian space schemes. The motion between the initial and final points is known at each instant of time and it is controllable. The motion is also visual so it is easy to describe. However, a singularity may occur at any point. We cannot be sure that there will be no singular in the trajectory.

3.4 Joint-Space Schemes

In this section, methods of path generation are described in terms of functions of joint angles. Each path point is actually a frame usually expressed in terms of a desired position and the orientation of the tool frame $\{T\}$, relative to the station frame $\{S\}$. Each of these via points is converted into a set of desired joint angles by applying inverse kinematics. Then a smooth function is found for each of the n joints that pass through via points and end at the final goal point. The elapsed time for each segment is the same for each joint so that all joints will reach via point at the same time. As a result, the desired Cartesian position of $\{T\}$ is satisfied at each via point.

Specifying the same duration for each joint, desired joint angle function for a particular joint does not depends on the functions for the other joints. As a result, joint-space schemes achieve the desired position and orientation at via points. Joint space schemes are the easiest to compute compared to Cartesian space schemes. Also there will not be any singularity problems for the delta robot mechanism.

3.4.1 Cubic Polynomials

In order to move the manipulator from its original position to a goal position at a certain rate, we have to know some equations to define a path. The set of joint angles is easily calculated with the help of inverse kinematic dynamics of the manipulator so we have the initial

angle and the goal angle of the manipulator. We try to find the function where the values are the initial joint angles at t_0 and where the values are the goal joint angles at t_f . This process is applied for joint space schemes.

Similarly, in Cartesian space scheme, we know the initial angles of the motors and the end-effector initial position is easily calculated with the help of forward kinematics. The goal position is the desired position to move. We try to find the function where the value of the function is the initial position at t_0 and where the value of the function is the goal position at t_f .

Position, velocity and acceleration in Cartesian space corresponds to angle, joint velocity and joint acceleration in joint space respectively. Therefore, the equations calculated for the joint space also applies to the Cartesian space. In this chapter, we will use position, velocity and acceleration terms for both the space schemes.

There are many smooth functions that are different than each other between two points. They are shown in Figure 3.2. To obtain a cubic polynomial, four constraints are needed. The initial and the goal position of the manipulator are two constraints:

$$\theta(0) = \theta_0 \quad , \quad \theta(t_f) = \theta_f \quad 3.1$$

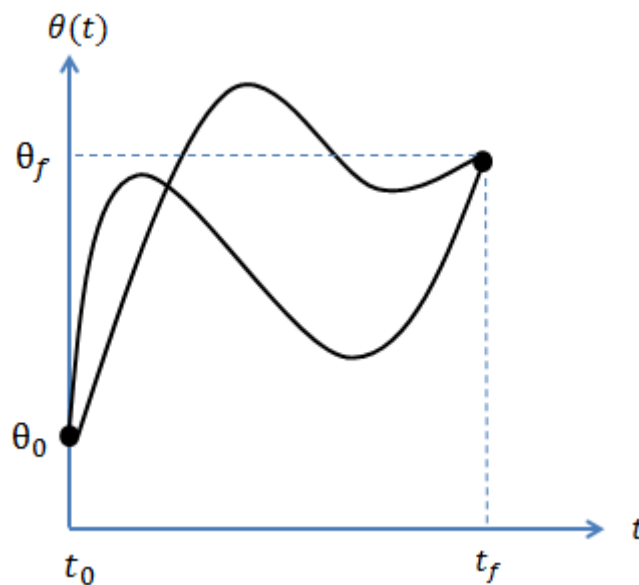


Figure 3.2: Possible Path Shapes for a Single Joint

The other constraints come from velocity. The final and the initial velocities have to be zero so our desired trajectory has to satisfy the following constraints:

$$\dot{\theta}(0) = 0 \quad , \quad \dot{\theta}(t_f) = 0 \quad 3.2$$

In order to satisfy Equation 3.1 and 3.2, the order of the trajectory function has to be a 3rd order polynomial function. These four constraints are used for solving four unknown coefficients of the function. The cubic polynomial has the following form:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 \quad 3.3$$

If the cubic polynomial is derived two times respect to time, we obtain velocity and acceleration of the manipulator:

$$\begin{aligned} \theta(t) &= a_0 + a_1 t + a_2 t^2 + a_3 t^3 \\ \dot{\theta}(t) &= a_1 + 2a_2 t + 3a_3 t^2 \\ \ddot{\theta}(t) &= 2a_2 + 6a_3 t \end{aligned} \quad 3.4$$

When four constraints are applied to the equation 3.3 and 3.4, we obtain:

$$\begin{aligned} \theta_0 &= a_0 = 0 \\ \theta_f &= a_0 + a_1 t_f + a_2 t_f^2 + a_3 t_f^3 \\ 0 &= a_1 \\ 0 &= a_1 + 2a_2 t_f + 3a_3 t_f^2 \end{aligned} \quad 3.5$$

Solving these equations for the coefficients of cubic polynomial, we also obtain:

$$\begin{aligned} a_0 &= \theta_0 \\ a_1 &= 0 \\ a_2 &= \frac{3}{t_f^2} (\theta_f - \theta_0) \\ a_3 &= -\frac{2}{t_f^3} (\theta_f - \theta_0) \end{aligned} \quad 3.6$$

3.4.1.1 In-Lab Exercise: Cubic polynomials for single point

1. A single-link robot is at $\theta = 0$ degrees. It is desired that the joint of the robot will move $\theta = 40$ degrees by the end of two seconds. Find the cubic polynomial coefficients.
2. Open 'TrajectoryGeneration.slx' that is shown in Figure 3.3 and configure the initial and the final theta values. Observe the graphs.

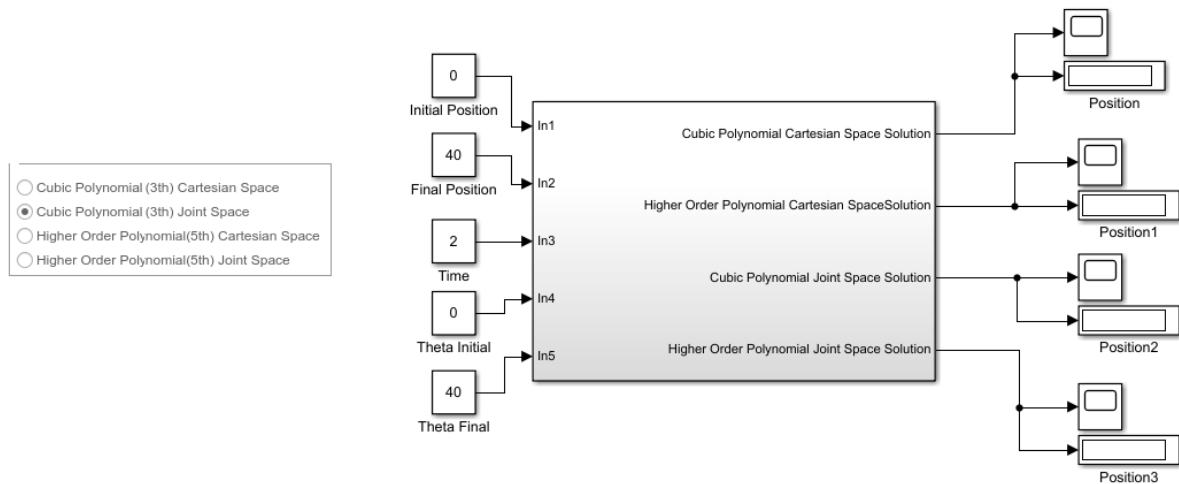


Figure 3.3: Block Diagram of "TrajectoryGeneration.slx"

3.4.1.2 In-Lab Answer: Cubic polynomials for single point

1. By using Equations 3.4 we can easily calculate the cubic polynomial coefficients:

$$\theta(t) = 30t^2 - 10t^3$$

$$\dot{\theta}(t) = 60t - 30t^2 \quad 3.7$$

$$\ddot{\theta}(t) = 60 - 60t$$

Coefficients of the cubic polynomial are:

$$a_0 = 0$$

$$a_1 = 0$$

$$a_2 = \frac{3}{4}(40 - 0) = 30 \quad 3.8$$

$$a_3 = -\frac{2}{8}(40 - 0) = -10$$

- Position, velocity and acceleration graphs are shown in Figure 3.4.

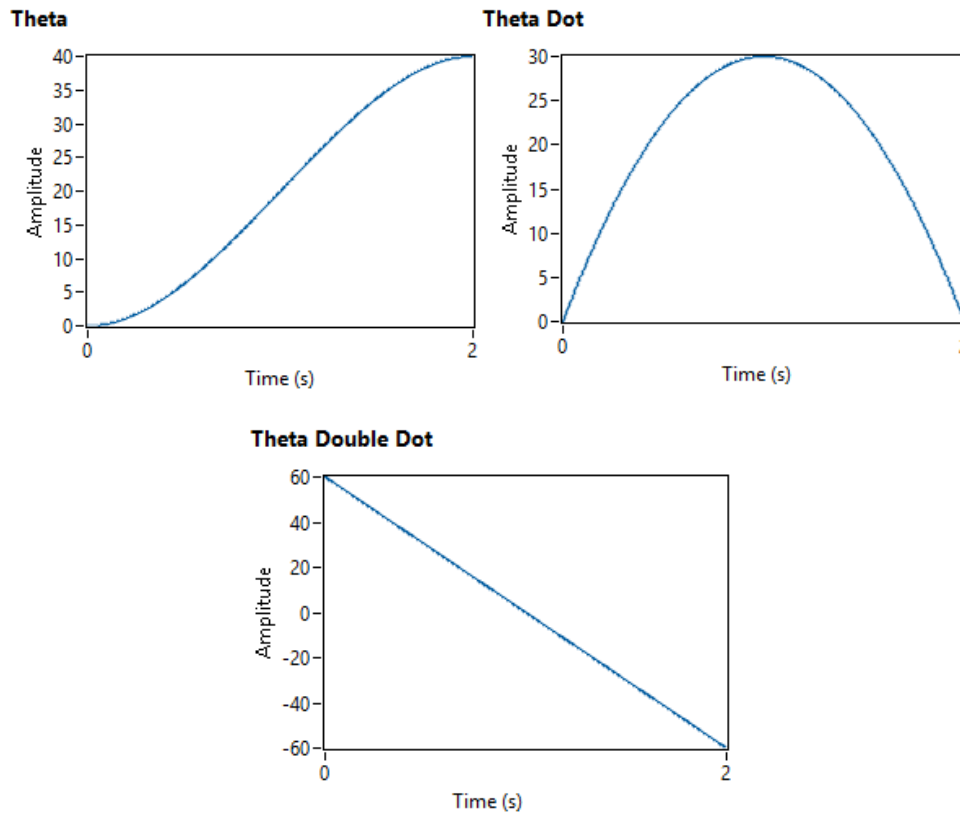


Figure 3.4: Angle, Angular Velocity and Angular Acceleration Profiles

3.4.1.3 In-Lab Exercise: Cubic polynomials for Cartesian Space

- Select Cubic Polynomial (3th) Cartesian Space from “TrajectoryGeneration.slx”.
- Enter the desired final positions and the final time. Run the program. Observe the graphs.

3.4.1.4 In-Lab Exercise: Cubic polynomials for Joint Space

- Cubic Polynomial (3th) Joint Space from “TrajectoryGeneration.slx”.
- Enter the desired final positions and the final time. Run the program. Observe the graphs.

3.4.2 Cubic Polynomials for a path with via points

There are some parameters that have to be decided for a trajectory to be calculated; for example, whether the motion should be linear, quadratic or cubic; or perhaps if the motion should be done with some predefined start and end velocity, or constant velocity or maybe

with minimum time. Then with this information and the help of interpolation, a trajectory can be calculated. This trajectory contains the information about the velocity, acceleration and position.

In the previous section, we have learned how to generate trajectories with cubic polynomials by using a desired duration and the initial and the final points. In general, we would like to add to the generated trajectories some special points in space, e.g. via points, where the end-effector passes. Cubic polynomials with zero velocity constraints cause the robot stop at each point. However, it is possible to define via points with non-zero velocity. If desired velocities of the joints at the via points are known, then we can construct cubic polynomials as before; now, however, the velocity constraints at each end are not zero, but rather, some known velocity. The Equation 3.2 becomes:

$$\dot{\theta}(0) = \dot{\theta}_0 \quad , \quad \dot{\theta}(t_f) = \dot{\theta}_f \quad 3.9$$

The four equations describing this general cubic are:

$$\begin{aligned} \theta_0 &= a_0 \\ \theta_f &= a_0 + a_1 t_f + a_2 t_f^2 + a_3 t_f^3 \\ \dot{\theta}_0 &= a_1 \\ \dot{\theta}_f &= a_1 + 2a_2 t_f + 3a_3 t_f^2 \end{aligned} \quad 3.10$$

Solving these equations for the a_i , we obtain:

$$\begin{aligned} a_0 &= \theta_0 \\ a_1 &= \dot{\theta}_0 \\ a_2 &= \frac{3}{t_f^2}(\theta_f - \theta_0) - \frac{2}{t_f}\dot{\theta}_0 - \frac{1}{t_f}\dot{\theta}_f \\ a_3 &= -\frac{2}{t_f^3}(\theta_f - \theta_0) + \frac{1}{t_f^2}(\dot{\theta}_f + \dot{\theta}_0) \end{aligned} \quad 3.11$$

Using Equation 3.10, we can calculate the cubic polynomial that connects any initial and final positions with any initial and final velocities. If we have the desired joint velocities at each via point, then we simply apply (3.10) to each segment to find the required cubic polynomials.

3.4.2.1 In-Lab Exercise: Cubic polynomials for a path with via points

Solve for the coefficients of two cubic polynomials that are connected in a two-segment spline with continuous acceleration at the intermediate via point. The initial angle is θ_0 , the via point is θ_v , and the goal point is θ_g .

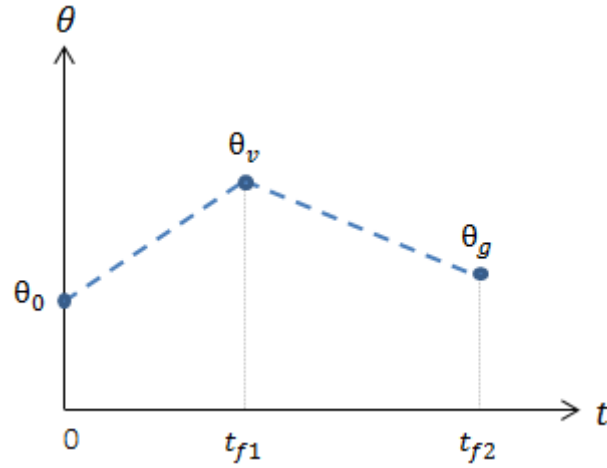


Figure 3.5: Initial, Via and Goal Points with Respect to Time

The first and second cubic polynomials are:

$$\begin{aligned}\theta_1(t) &= a_{10} + a_{11}t + a_{12}t^2 + a_{13}t^3 \\ \theta_2(t) &= a_{20} + a_{21}t + a_{22}t^2 + a_{23}t^3\end{aligned}\tag{3.12}$$

Each cubic will be evaluated over an interval starting at $t = 0$ and ending at $t = t_{fi}$, where $i = 1$ or $i = 2$.

The constraints we wish to enforce are:

$$\begin{aligned}\theta_0 &= \theta_{10} \\ \theta_v &= a_{10} + a_{11}t_{f1} + a_{12}t_{f1}^2 + a_{13}t_{f1}^3 \\ \theta_v &= a_{20} \\ \theta_g &= a_{20} + a_{21}t_{f2} + a_{22}t_{f2}^2 + a_{23}t_{f2}^3 \\ 0 &= a_{11} \\ 0 &= a_{21} + 2a_{22}t_{f2} + 3a_{23}t_{f2}^2 \\ a_{11} + a_{12}t_{f2} + a_{13}t_{f1}^2 &= a_{21} \\ 2a_{12} + 6a_{13}t_{f1} &= a_{21}\end{aligned}\tag{3.13}$$

These constraints specify a linear-equation problem having eight equations and eight unknowns. Solving for the case $t_f = t_{f1} = t_{f2}$, we obtain:

$$\begin{aligned}
a_{10} &= \theta_0 \\
a_{11} &= 0 \\
a_{12} &= \frac{12\theta_v - 3\theta_g - 9\theta_0}{4t_f^2} \\
a_{13} &= \frac{-8\theta_v + 3\theta_g + 5\theta_0}{4t_f^3} \\
a_{20} &= \theta_v \\
a_{21} &= \frac{3\theta_g - 3\theta_0}{4t_f} \\
a_{22} &= \frac{-12\theta_v + 6\theta_g + 6\theta_0}{4t_f^2} \\
a_{23} &= \frac{8\theta_v - 5\theta_g - 5\theta_0}{4t_f^3}
\end{aligned} \tag{3.14}$$

3.4.3 Higher Order Polynomials

In the previous section, we've seen how to build cubic polynomials with via points but the problem is at each via point acceleration suddenly changes and this causes jerks in the system. To prevent these disturbances, we can use higher order polynomials where we can specify the acceleration along with position and velocity. To do this a quintic polynomial is required where the general notation is:

$$\theta(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 \tag{3.15}$$

Where the constraints are given as:

$$\begin{aligned}
\theta_0 &= a_0 \\
\theta_f &= a_0 + a_1t_f + a_2t_f^2 + a_3t_f^3 + a_4t_f^4 + a_5t_f^5 \\
\dot{\theta}_0 &= a_1 \\
\dot{\theta}_f &= a_1 + 2a_2t_f + 3a_3t_f^2 + 4a_4t_f^3 + 5a_5t_f^4 \\
\ddot{\theta}_0 &= 2a_2 \\
\ddot{\theta}_f &= 2a_2 + 6a_3t_f + 12a_4t_f^2 + 20a_5t_f^3
\end{aligned} \tag{3.16}$$

These constraints specify a linear set of six equations with six unknowns, whose solution is:

$$a_0 = \theta_0 \tag{3.17}$$

$$a_1 = \dot{\theta}_0$$

$$a_2 = \frac{\ddot{\theta}_0}{2}$$

$$a_3 = \frac{20\theta_f - 20\theta_0 - (8\dot{\theta}_f + 12\dot{\theta}_0)t_f - (3\ddot{\theta}_0 - \ddot{\theta}_f)t_f^2}{2t_f^3}$$

$$a_4 = \frac{30\theta_0 - 30\theta_f - (14\dot{\theta}_f + 16\dot{\theta}_0)t_f + (3\ddot{\theta}_0 - 2\ddot{\theta}_f)t_f^2}{2t_f^4}$$

$$a_5 = \frac{12\theta_f - 12\theta_0 - (6\dot{\theta}_f + 6\dot{\theta}_0)t_f - (\ddot{\theta}_0 - \ddot{\theta}_f)t_f^2}{2t_f^5}$$

3.4.3.1 In-Lab Exercise: Higher order polynomials for Cartesian space

1. Select Higher Order Polynomial (5th) Cartesian Space from “TrajectoryGeneration.slx”.
2. Enter the desired final positions and the final time. Run the program. Observe the graphs.

3.4.3.2 In-Lab Exercise: Higher order polynomials for Joint Space

1. Select Higher Order Polynomial (5th) Cartesian Space from “TrajectoryGeneration.slx”
2. Enter the desired final positions and the final time. Run the program. Observe the graphs.

3.4.4 Linear Function with Parabolic Blends

In this chapter, there are three segments for a linear function with parabolic blends. The first and third segments are interpolated with a parabola. The first segment specifies trajectory from the initial position to the blend point. The third segment specifies trajectory from second blend point to the final point. In this way, the discontinuity of the velocity at the beginning and at the end of the motion is prevented. The second segment is interpolated as a linear position or a constant velocity that is between two blends which is shown in Figure 3.6.

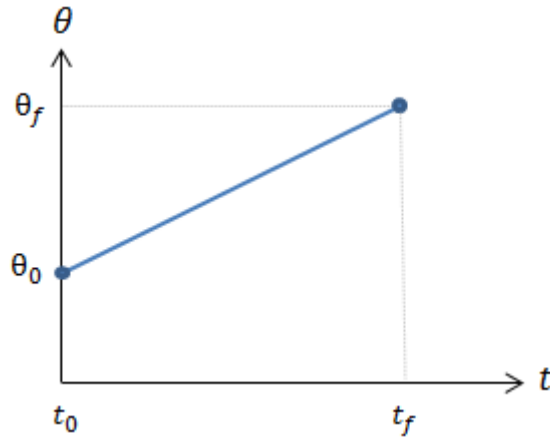


Figure 3.6: The Second Segment with Linear Position

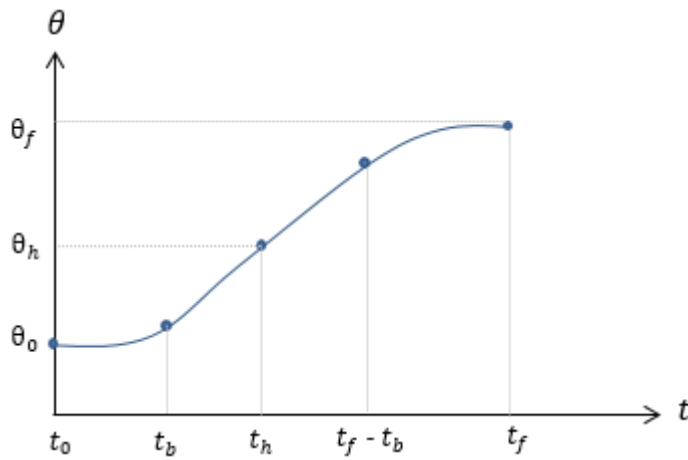


Figure 3.7: Linear Segment with Parabolic Blends

Before starting to calculations, we assumed that the parabolic blends have the same duration and the acceleration is same and constant during blends. In addition, the curve is symmetric about the halfway point in time, t_h , and about the halfway point in position, θ_h . The velocity at the end of the blend region must equal to the velocity of the linear section, thus the velocity:

$$\ddot{\theta}_b t_b = \frac{\theta_h - \theta_b}{t_h - t_b} \quad 3.18$$

θ_b is the angle at the end of the blend region and $\ddot{\theta}_b$ is the acceleration acting during the blend region. θ_b is:

$$\theta_b = \theta_0 + \frac{1}{2} \ddot{\theta}_b t_b^2 \quad 3.19$$

$t = 2t_h$ where t is the desired duration of the motion. Combining Equation 3.18 and 3.19, we obtain:

$$\ddot{\theta}_b t_b^2 - \ddot{\theta}_b t t_b + (\theta_f - \theta_0) = 0 \quad 3.20$$

Then solving 3.20 for t_b in terms of known parameters, we also obtain:

$$t_b = \frac{t}{2} - \frac{\sqrt{\ddot{\theta}_b^2 t^2 - 4\ddot{\theta}(\theta_f - \theta_0)}}{2\ddot{\theta}_b} \quad 3.21$$

There is a constraint on the acceleration which is:

$$\ddot{\theta} \geq \frac{4(\theta_f - \theta_0)}{t^2} \quad 3.22$$

3.4.5 Linear function with parabolic blends for a path with via points

In this part, we consider the linear paths with parabolic blends for the case where there are arbitrary number of via points. The Figure 3.8 shows a set of joint-space via points where the linear functions connect via points and at each via point, parabolic blends are added.

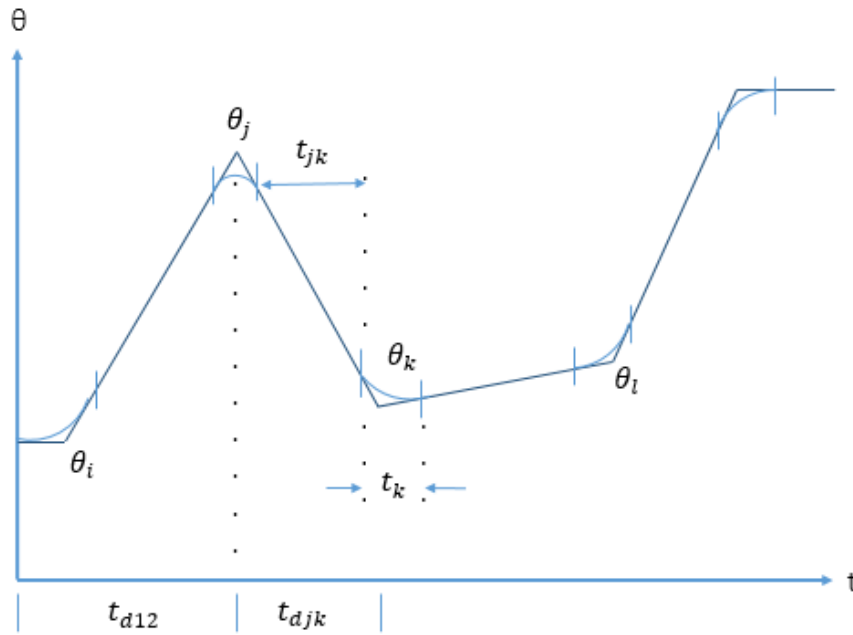


Figure 3.8: Linear Segment with Parabolic Blends

As the notations used according to the graph; the neighbor points called as j , k and l . The duration of the linear region between j and k points is t_{jk} , the duration of blend area at point k is t_k , the total duration of between j and k points t_{djk} . The velocity of the linear region between j and k points is $\dot{\theta}_{jk}$, the acceleration during the blend at point j is $\ddot{\theta}_j$. For the single segment case, there are numerous solutions that depend on the acceleration of the blends. Given the path points θ_k , the desired time t_{djk} and the magnitude of acceleration at each blend point $|\ddot{\theta}_k|$, we can calculate the blend times t_k .

$$\begin{aligned}
\dot{\theta}_{jk} &= \frac{\theta_k - \theta_j}{t_{djk}} \\
\ddot{\theta}_k &= \text{sgn}(\dot{\theta}_{kl} - \dot{\theta}_{jk})|\ddot{\theta}_k| \\
t_k &= \frac{\dot{\theta}_{kl} - \dot{\theta}_{jk}}{\ddot{\theta}_k} \\
t_{jk} &= t_{djk} - \frac{t_j}{2} - \frac{t_k}{2}
\end{aligned} \tag{3.23}$$

The first and last segments must be dealt with differently, because an entire blend region at one of the segments must be counted in the total segment time. For the first segment, we solve for t_1 by equating two expressions for the velocity during the linear part.

$$\frac{\theta_2 - \theta_1}{t_{12} - t_1/2} = \ddot{\theta}_1 t_1 \tag{3.24}$$

This can be solved for t_1 , the blend time at the initial point and then $\dot{\theta}_{12}$ and t_{12} can be computed as follows:

$$\begin{aligned}
\ddot{\theta}_1 &= \text{sgn}(\theta_2 - \theta_1)|\ddot{\theta}_1| \\
t_1 &= t_{d12} - \sqrt{t_{d12}^2 - \frac{2(\theta_2 - \theta_1)}{\ddot{\theta}_1}} \\
\dot{\theta}_{12} &= \frac{(\theta_2 - \theta_1)}{t_{d12} - t_1/2} \\
t_{12} &= t_{d12} - t_1 - \frac{t_2}{2}
\end{aligned} \tag{3.25}$$

The same procedure for the last segment can be followed like as:

$$\begin{aligned}
\frac{\theta_{n-1} - \theta_n}{t_{d(n-1)n} - t_n/2} &= \ddot{\theta}_n t_n \\
\ddot{\theta}_n &= \text{sgn}(\theta_{n-1} - \theta_n)|\ddot{\theta}_n| \\
t_n &= t_{d(n-1)n} - \sqrt{t_{d(n-1)n}^2 + \frac{2(\theta_n - \theta_{n-1})}{\ddot{\theta}_n}} \\
\dot{\theta}_{(n-1)n} &= \frac{(\theta_n - \theta_{n-1})}{t_{d(n-1)n} - t_n/2} \\
t_{(n-1)n} &= t_{d(n-1)n} - t_n - \frac{t_{n-1}}{2}
\end{aligned} \tag{3.26}$$

The blend times and velocities for a multi-segment path can be found following the above procedure. Generally, the user specifies only via points and the desired duration of the segments. And also, the system uses default values for acceleration for the joints.

3.4.5.1 In-Lab Exercise: Linear function with parabolic blends for a path with via points

The trajectory of a particular joint is specified as follows; path points in degrees: 15,30,20,15. The duration of these three segments should be respectively 2.0, 1.0, 3.0 seconds. The magnitude of the default acceleration to be used at all blend points is 40 degree/sec². Calculate all the segment velocities, the blend times and the linear times.

$$\ddot{\theta}_1 = 40$$

The blend time at the initial point:

$$t_1 = 2 - \sqrt{4 - \frac{2(30 - 15)}{40}} = 0.1972$$

$$\dot{\theta}_{12} = \frac{(30 - 15)}{2 - 0.1972/2} = 7.889$$

The velocity $\dot{\theta}_{23}$ (Eq. 3.23) is calculated as:

$$\dot{\theta}_{23} = \frac{20 - 30}{1} = -10$$

Also, the blend velocity and times can be found as (Eq. 3.23);

$$\ddot{\theta}_2 = \text{sgn}(\dot{\theta}_{23} - \dot{\theta}_{12})|\ddot{\theta}_2| = -40$$

$$t_2 = \frac{-10 - 7.889}{-40} = 0.4472$$

The duration of the linear part of the segment 1 is computed as:

$$t_{12} = 2 - 0.1972 - \frac{0.4472}{2} = 1.5792$$

After this point,

$$\ddot{\theta}_4 = 40$$

For the last segment, t_4 is computed as:

$$t_4 = 3 - \sqrt{9 + \frac{2(15 - 25)}{40}} = 0.0845$$

The velocity $\dot{\theta}_{34}$ (Eq. 3.26) is calculated as:

$$\dot{\theta}_{34} = \frac{15 - 25}{3 - 0.042} = -3.38$$

The acceleration $\ddot{\theta}_3$ and the blend time t_3 can be computed as:

$$\ddot{\theta}_3 = 40$$

$$t_3 = \frac{-3.38 + 10}{40} = 0.1655$$

Finally, we compute the linear duration of t_{23} and t_{34} :

$$t_{23} = 1 - \frac{0.4472}{2} - \frac{0.1655}{2} = 0.6936$$

$$t_{34} = 3 - \frac{0.1655}{2} - 0.0845 = 2.8327$$

4 Vision Guidelines for Delta Robot

4.1 Introduction to Vision Systems

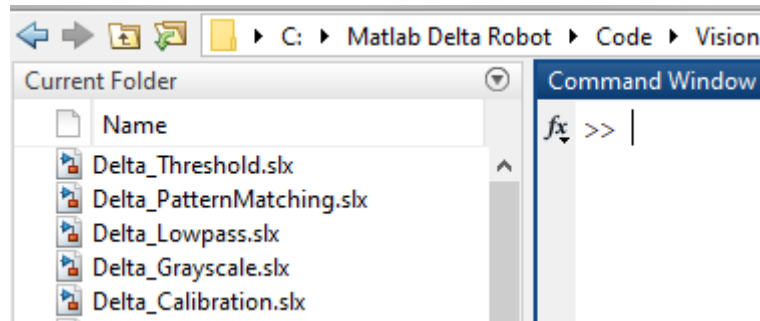


Figure 4.1: Vision Folder

The filters and operations used in image processing using the camera on the delta robot are described in this chapter. A Delta Robot application (Smash) using the exercises described in this chapter is shown at the end of the chapter.

In order to run the simulink models in the Vision folder on the Matlab platform, you need to install the “Matlab Support Package for USB Webcams” add-on from the Add-On Explorer.

"Matlab Support Package for USB Webcams" add-on is shown in Figure 4.2.

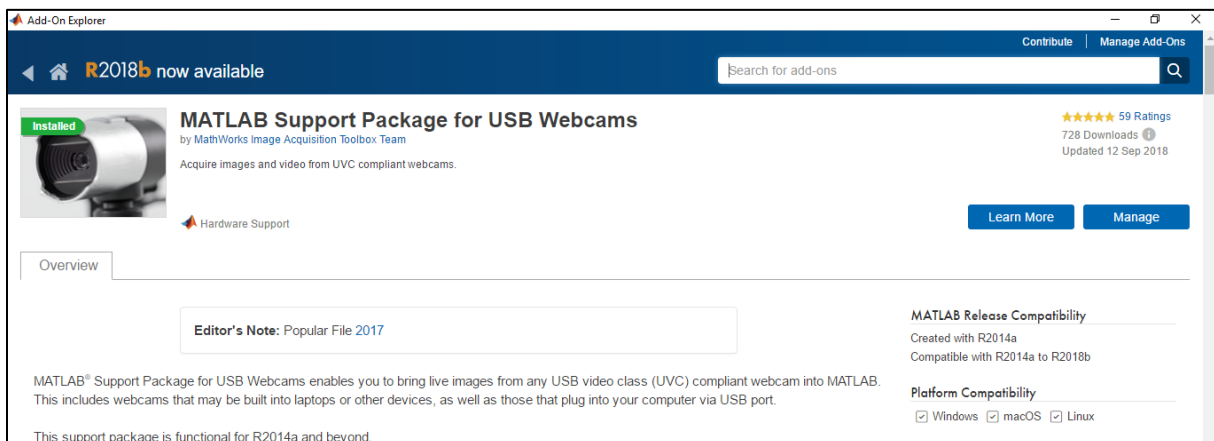


Figure 4.2: Matlab Support Package for USB Webcams

4.1.1 In-Lab Exercise: Vision Acquisition and Grayscale Processing

1. Open the “Delta_Grayscale.slx” under “Vision” folder.

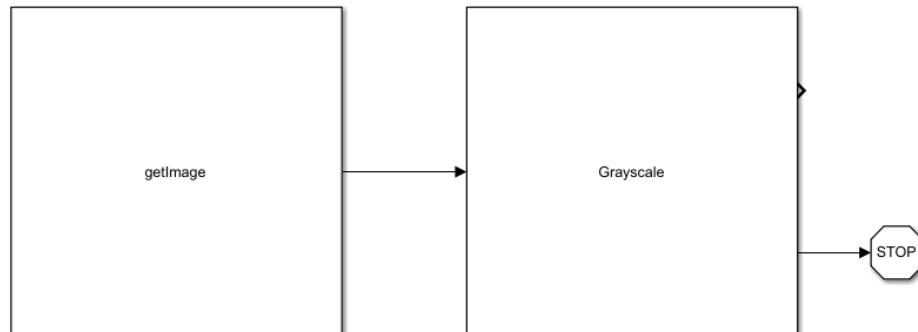


Figure 4.3: Block Diagram of “Delta_Grayscale.slx”

2. Plug the webcam cable into the USB slot of your computer.
3. Click the Run to run the program.
4. Wait for the Figure Windows open.
5. You will see two Figure which is shown in figure 4.4. Figure 1 is the original image and Figure 2 is the grayscale image.

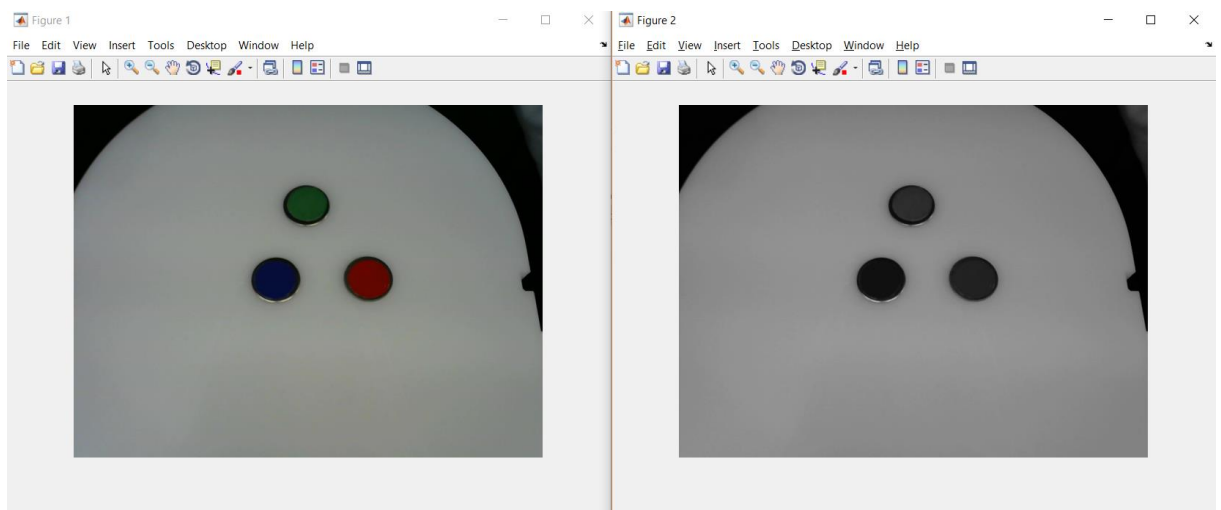


Figure 4.4: Grayscale Filter

4.1.2 Threshold

Threshold function converts a grayscale image to a binary image. In this way, the function basically performs an isolation that separates the object of interest from the background if there is enough contrast between the object and background. Applying threshold to an image with high contrast between objects and background separates the objects and the background. With this operation, background pixels are set to 0 and the object pixels are set to 1.

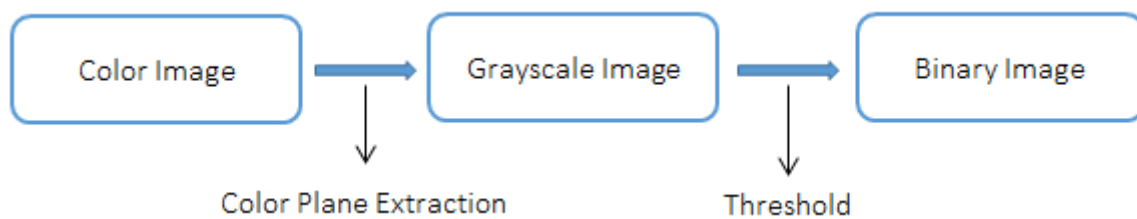


Figure 4.5: Process of Conversion Color Image to Binary Image

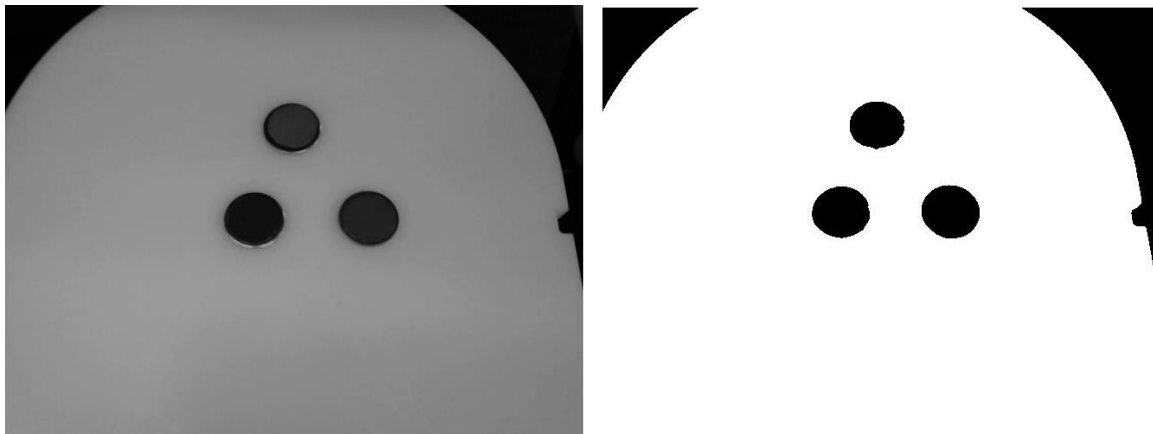


Figure 4.6: Threshold Process for Grayscale Image

1. Open the “Delta_Threshold.slx” under “Vision” folder.
2. Plug the webcam cable into the USB slot of your computer.
3. Click the Run to run the program.
4. Wait for the Figure Windows open.
5. You will see two Figure which is shown in figure 4.7. Figure 1 is the original image and Figure 2 is the grayscale image and Figure 3 is the threshold image.

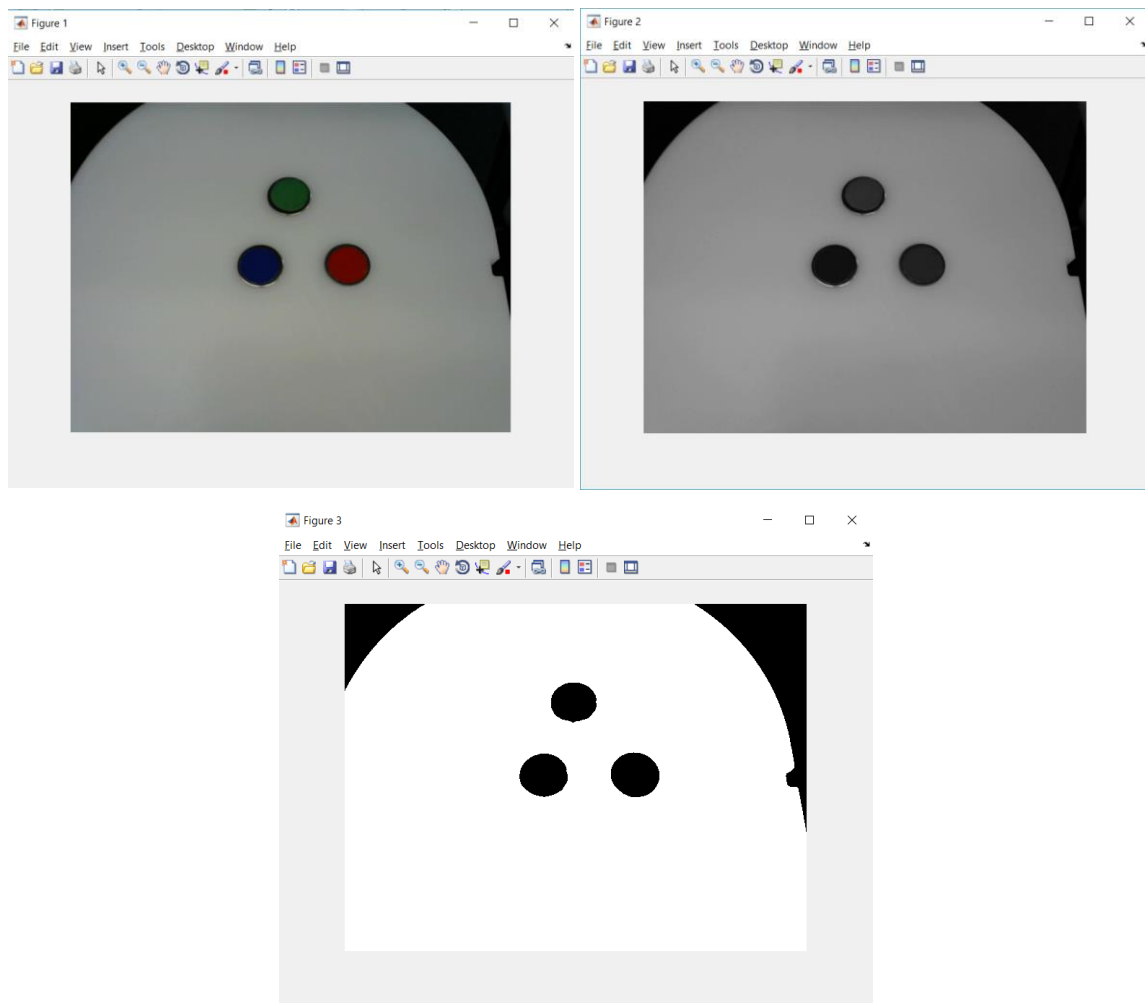


Figure 4.7: Obtain binary image

4.1.3 Pattern Matching

This algorithm measures the similarity between the template and the image. The weight of the similarity is in the range of 0-100. The minimum score can be changed by the user. Template image for coin is shown in Figure 4.8.

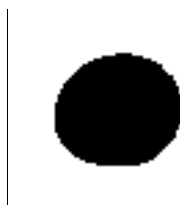


Figure 4.8: Template Image for Pattern Matching

4.1.3.1 In-Lab Exercise: Non-calibrated Pattern Matching

1. Open the “Delta_PatternMatching.slx” under “Vision” folder.

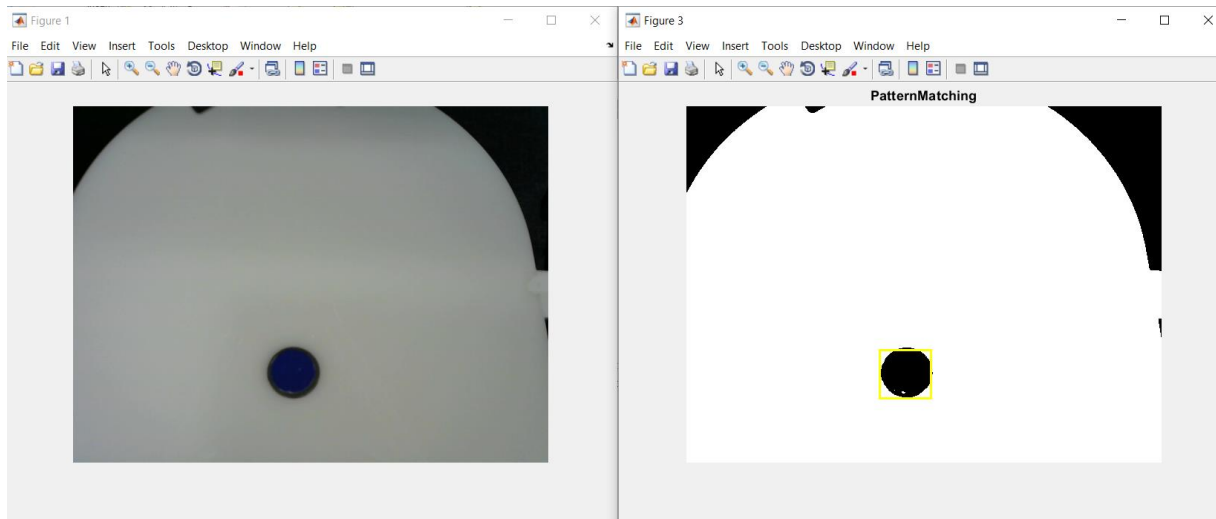


Figure 4.9: Front Panel of “Non-calibrated Pattern Matching”

2. Plug the webcam cable into the USB slot of your computer.
3. Run the program and wait for complete.
4. Acquire the images and examine

The Delta Robot’s camera is not perpendicular to object’s surface due to Delta Robot’s design. As a result, positions of the coins are not correct because of the camera’s distortion. To eliminate this error, we will add Image Calibration function in the next step.

4.1.3.2 Image Calibration

An image data is read in the form of pixels. Spatial calibration offers you to convert a measurement from pixel units into physical units. If the camera is perpendicular to the image plane, one pixel corresponds to one inch. In general, the camera takes the images at a certain angle and is not perpendicular. Therefore, distorted images are acquired. To compensate the perspective errors and nonlinear lens distortion of the camera, the image calibration function should be used.

1. First, open and run “DeltaCalibration.slx” under the Smash or PickandPlace folder. The end effector position is set to zero in x and y direction. This is an initial position of Delta Robot. Calibration paper has also a black circle that shows the initial position of Delta Robot.

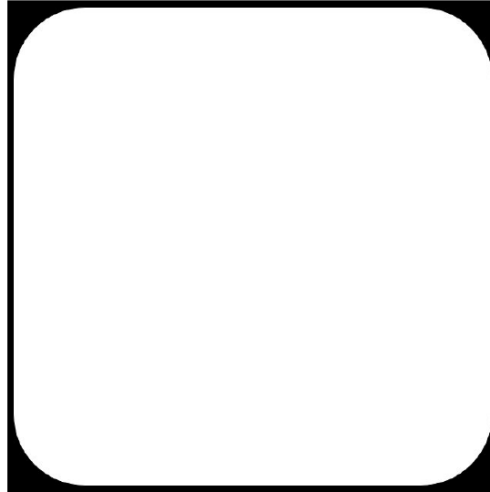


Figure 4.10: The Calibration Paper

2. Select “Center” from Calibration Points.
3. Now, locate the calibration paper on your image plane in Figure 4.10 and decrease the z position until the end effector overlaps the black circle.
4. Select “Y-Down” and set Z Value to -140 and close the program.
5. Set current folder “Vision” in MATLAB. Open “Delta_Calibration.slx” and run it. Two figures will be opened. You should take a screenshot from “Non-Calibrated Image” and save it to Vision folder as “calibpaper.jpg”. You should copy this image to Smash and PickandPlace folder too.

4.1.3.2.1 In-Lab Exercise: Vision Acquisition with Calibration & Correction

1. Open the “Delta_Calibration.slx” under “Vision” folder.

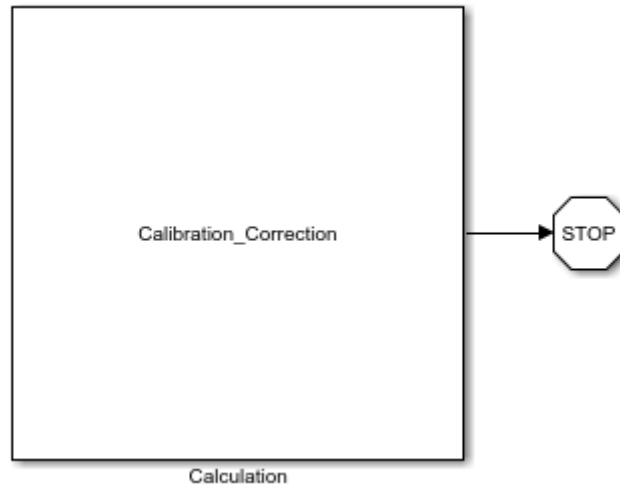


Figure 4.11: Block Diagram of “Delta_Calibration.slx”

2. Plug the webcam cable into the USB slot of your computer.
3. Run the program. Acquire the images. As shown in figure 4.12, this block calibrates and corrects the image according to the image taken with the calibration paper and crops the image according to the end-effector's working area.

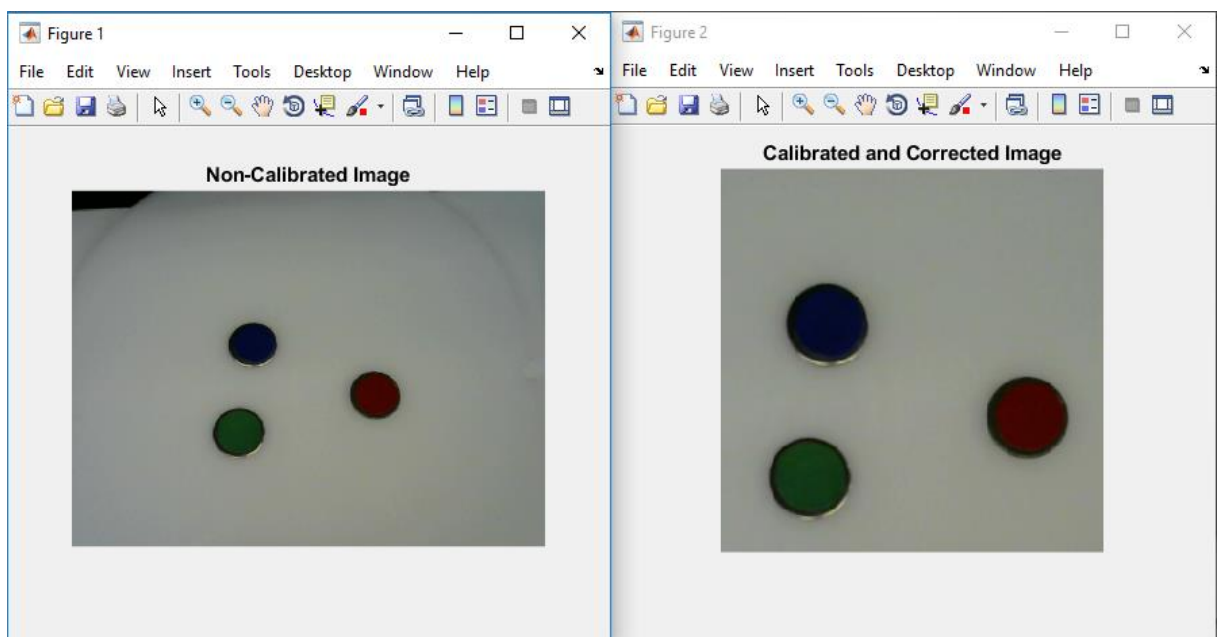


Figure 4.12: Non-Calibrated and Calibrated Images

4.1.3.2.2 In-Lab Exercise: Calibrated Pattern Matching

1. Open the “Calibrated_Delta_PatternMatching.slx” under “Vision” folder. Run the program and examine the figures.

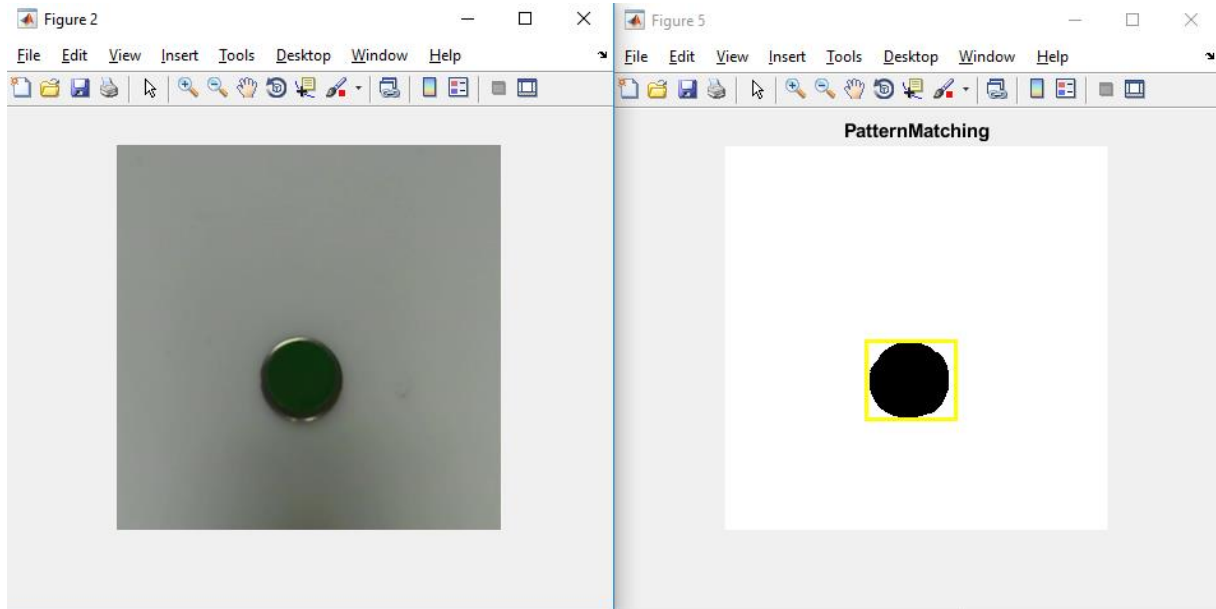


Figure 4.13: Pattern Matching Figures

2. This program combines all the image processing steps. First, we acquire the color image and convert into a grayscale image. Then, image calibration is applied to the grayscale image. Calibrated grayscale image is converted into a binary images using Threshold. Finally, pattern matching algorithm is applied to the binary image and image processing is complete. The detection of the coins has been completed successfully.

4.2 Pick and Place Application of the Delta Robot

Pick and Place application requires two different Simulink model. “PickandPlacePCSide.slx” runs on computer and acquires the image and detects the object positions. Position of the object is sent to Arduino. “PickandPlaceRobotSide.slx” reads the position. A trajectory is generated between the initial and the final positions on Arduino side and three motors move the end-effector to the object position. Then, the object is grabbed and The motors of the Delta robot move towards where the object is to be released. After the object is released, the motors move to the other object. When objects are collected, the program stops.

4.2.1 In-Lab Exercise: PickandPlace Application

1. Open the “PickandPlaceRobotSide.slx” Simulink model

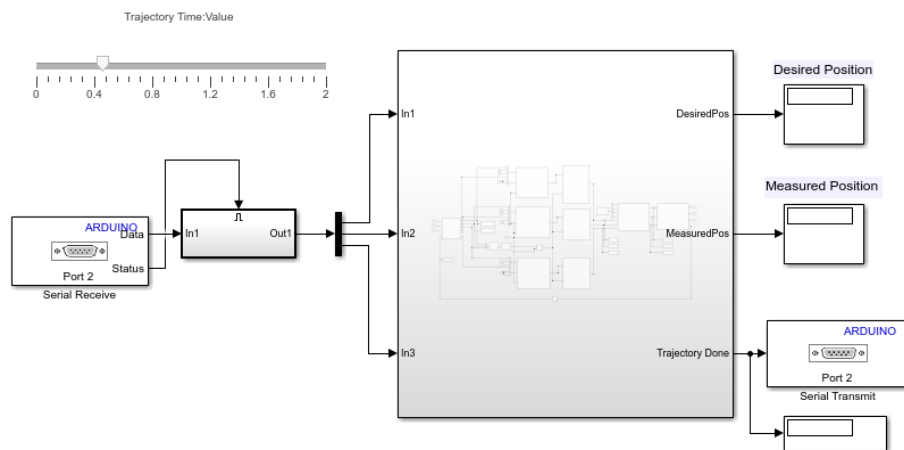


Figure 4.14: “PickandPlaceRobotSide.slx”

2. Open the “PickandPlacePCSide.slx”

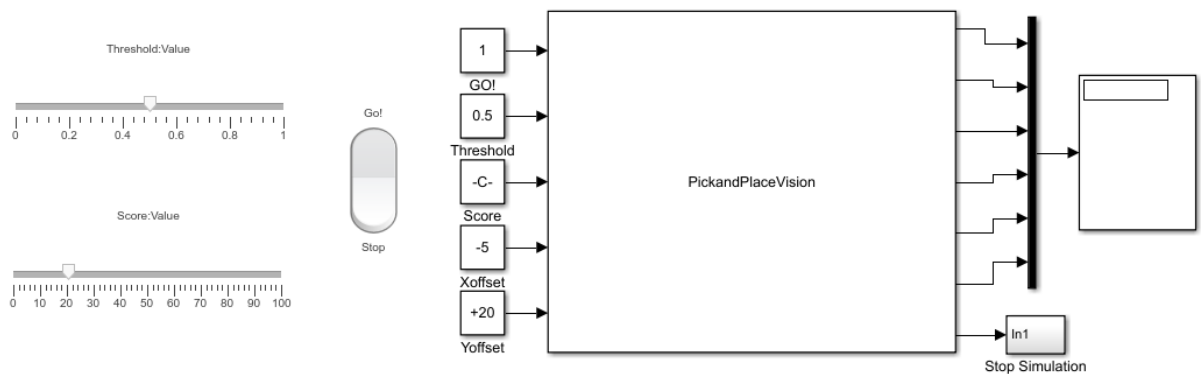


Figure 4.15: “PickandPlacePCSide.slx”

3. Double-click the PickandPlaceVision block in the “PickandPlacePCSide.slx”. simulink model before running the programs. Change the comport number of the USB to TTL module in the window that opens. This window is shown in figure 4.16. Use the “Device Manager” to find the USB to TTL comport number.

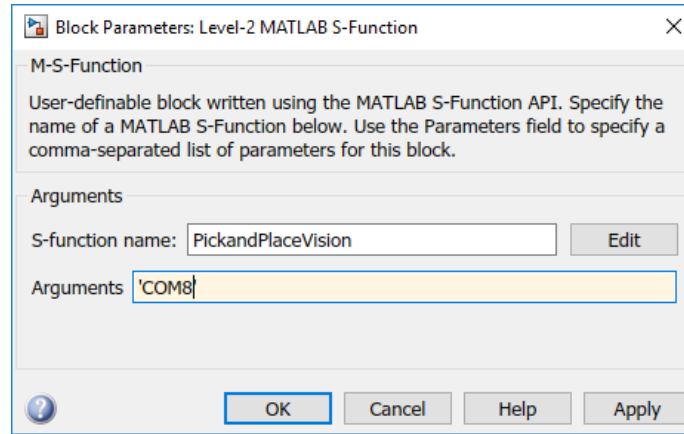


Figure 4.16: Comport number of USB to TTL Module

4. Be sure to perform height calibration correctly before operating this application. Please place the coins in the workspace of the Delta Robot.
5. First, run “PickandPlaceRobotSide.slx”. Then, run the “PickandPlacePCSide.slx”. The Delta Robot moves to the initial position and acquire the image.

Note: You can change the threshold value according to the light of the environment.

Camera detects the object and motors move the end-effector to the object positions. End-effector grabs the object and moves towards where the object is to be released.

6. Sometimes the detected pixel values are shifted due to mechanical assembly. To fix this problem, the Xoffset and Yoffset values in the “PickandPlacePCSide.slx” simulink model must be set correctly
7. To close the application properly, first stop the “SmashHostSide2017b.slx” program.

4.3 Smash Application of the Delta Robot

Smash application requires two different Simulink model. “SmashHostSide2017b.slx” runs on My Computer and acquires the image and detects the object positions. Position of the object is sent to Arduino. “SmashArduinoSide2017b.slx” reads the position. A trajectory is generated between the initial and the final positions on Arduino side and three motors move the end-effector to the object position. Then, the object is touched and three motors move the end-effector to the initial position. Finally, Arduino sends the “Trajectory Done” function to continue the Smash application.

4.3.1 In-Lab Exercise: Smash Application

1. Open the “SmashArduinoSide2017b.slx” Simulink model.

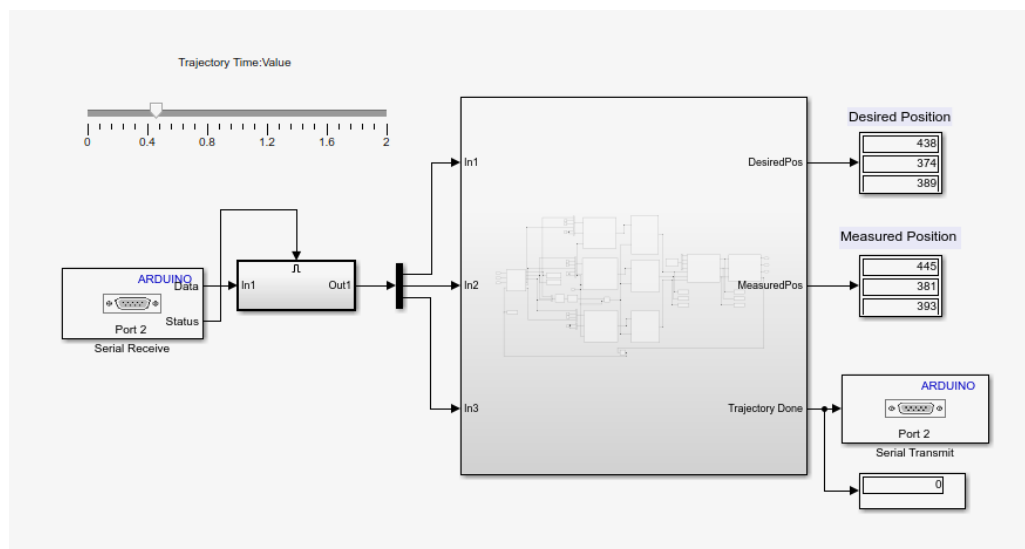


Figure 4.14: Block Diagram of “SmashArduinoSide2017b.slx”

2. Open the “SmashHostSide2017b.slx”.

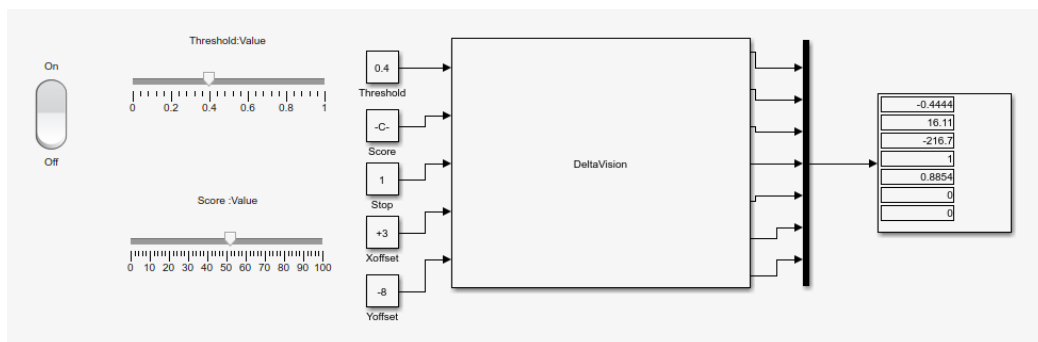


Figure 4.15: Block Diagram of “SmashHostSide2017b.slx”

3. Double-click the DeltaVision block in the “SmashHostSide2017b.slx”. simulink model before running the programs. Change the USB to TTL module’s comport number in the window that opens. This window is shown in figure 4.16. Use the “Device Manager” to find the USB to TTL comport number.

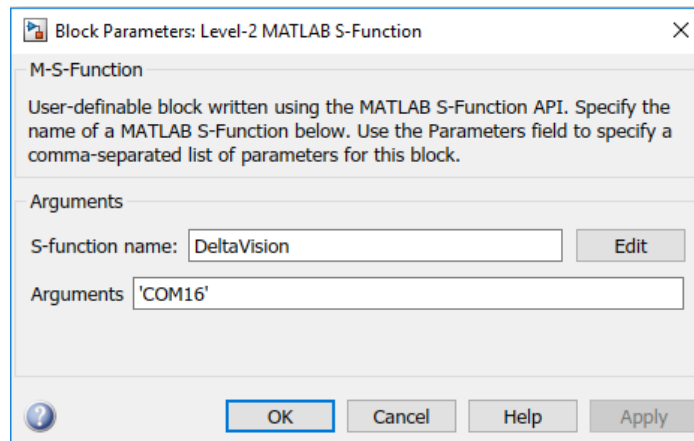


Figure 4.16: Comport number of USB to TTL Module

4. Be sure to perform height calibration correctly before operating this application.
5. First, run “SmashArduinoSide2017b.slx”. Then, run the “SmashHostSide2017b.slx”. The Delta Robot moves to the initial position and acquire the image.
6. You can change the threshold value according to the light of the environment. The value in the 5th row from the values displayed at the block output is the score value of the matching pattern. According to this value, you can set the desired score value in the input.
7. Open **Smash Lights app** on tablet/phone and locate the tablet/phone under the camera and press “Start” button at “SmashHostSide2017b.slx”
8. Sometimes the detected pixel values are shifted due to mechanical assembly. To fix this problem, the Xoffset and Yoffset values in the SmashHostSide.slx simulink model must be set correctly
Camera detects the object and motors move the end-effector to the object positions. End-effector touches the object and moves to the initial position. Smash application restarts after “Trajectory Done” is sent.
9. To close the application properly, first stop the “SmashHostSide2017b.slx” program.

References

- [1] John J. Craig. Introduction to Robotics: Mechanics and Control (3rd Edition). Prentice Hall, 3 edition, August 2004.
- [2] Alashqar, A.H. (2007). Modeling and High Precision Motion Control of 3 DOF Parallel Delta Robot Manipulator. Post Graduated Thesis, The Islamic University of Gaza, Palestine.
- [3] Trossen Robotic Community (t.y.), <<http://forums.trossenrobotics.com/tutorials/introduction-129/delta-robot>>, (ET: 10.06.2015)
- [4] R.L. Williams II, "The Delta Parallel Robot: Kinematics Solutions", Internet Publication, www.ohio.edu/people/williar4/html/pdf/DeltaKin.pdf, April 2015.
- [5] Olsson A., Modeling and control of a Delta-3 robot. Master Thesis, Lund University, February 2009.
- [6] K.S. Hsu, M. Karkoub, M.C. Tsai and M.G. Her, "Modelling and index analysis of a Delta-type mechanism", 2004.
- [7] S. Folea, "Practical Applications and Solutions Using LabVIEW Software", <intechweb.org>
- [8] R. Posada-Gómez, O.O. Sandoval-González, A. M. Sibaja etc., Instituto Tecnológico de Orizaba, Departamento de Postgrado e Investigación, México